

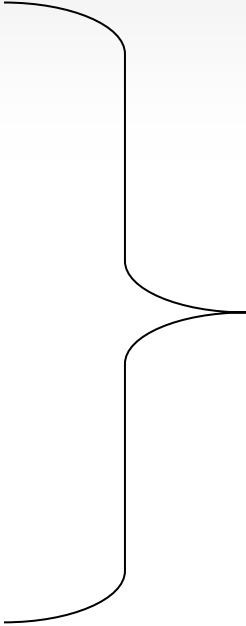


物理建模

计算机系 张松海

建模包括什么？

- 物体几何属性
 - 基于几何的方法
 - 基于图像的方法
- 物体的物理属性
- 物体的行为属性
 - 动作（人）



对象模型

1935 Kapla Planks
File Size: 121Mb
Render Time: 40s per frame
Simulation Time: 20m

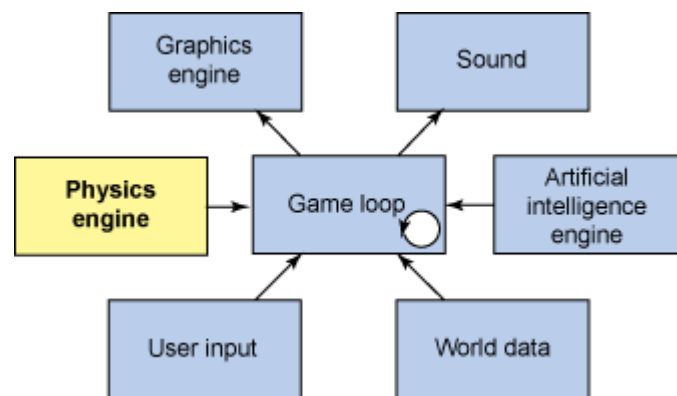
物理模拟



NVIDIA DRIVE Sim

物理建模

- 建立对象物体的某些物理模型，使场景对象遵循客观的物理规律，从而生成具有一定真实感和逼真性的三维场景
- 常见的力学规律
 - 刚体动力学
 - 弹性力学
 - 塑性力学
 - 断裂力学
 - 流体力学（气体、液体）



物理引擎

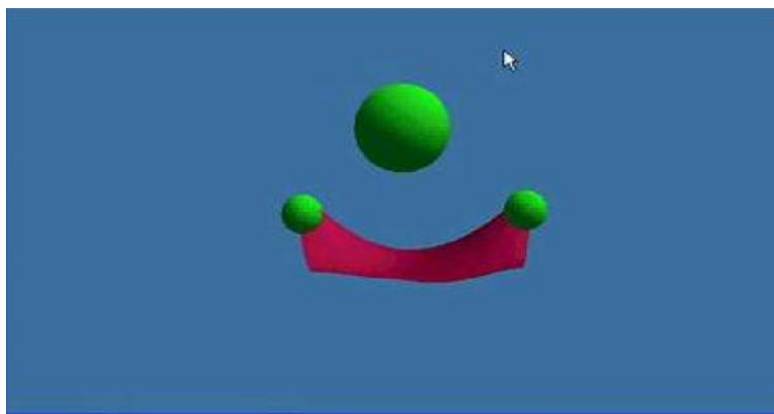
目标：实时近似

物理引擎

- 最简单的定义：物理引擎就是一个开发包。
- 在计算机绘制的虚拟场景中，物理引擎通过为场景中的各种物体赋予真实的物理属性来模拟计算它们的运动、旋转、碰撞反应和其它相互作用。
- 物理引擎使用对象物理属性（动量、扭矩或者弹性）来模拟物体行为，这不仅可以得到更加真实的结果，而且形成的通用工具集使开发人员更加容易掌握。

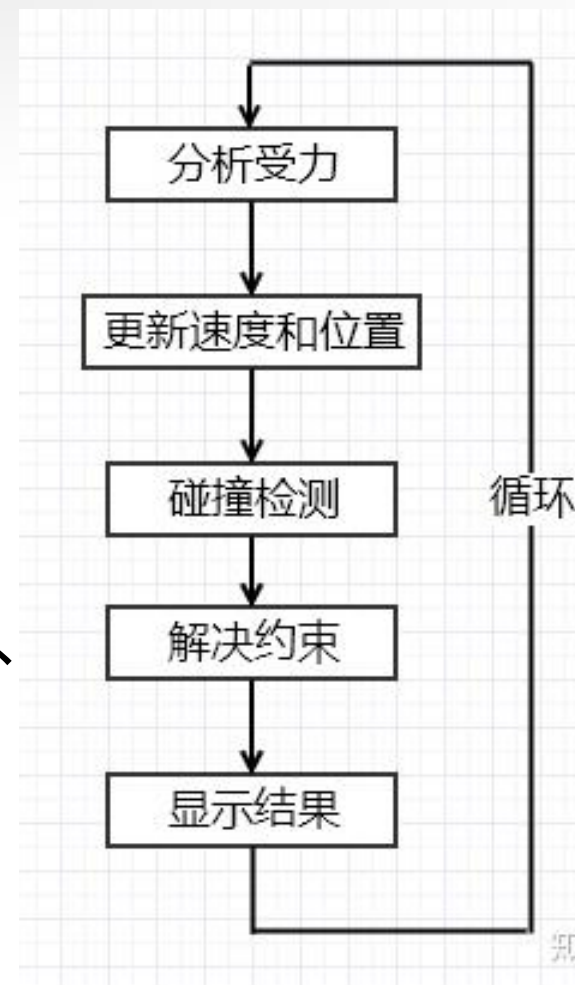
物理引擎的效果

- 在物理引擎的支持下，VR场景中的模型有了实体，一个物体可以具有质量、可以受到重力、可以落在地面上、可以和别的物体发生碰撞、可以受到用户施加的推力、可以因为压力而变形、可以有液体在表面上流动。



物理引擎的内容

- 物理引擎技术涉及到计算机学科的诸多领域，包括：计算机算法、计算机图形学、数值计算、计算几何等等。
- 引擎中包含的处理模块包括：碰撞检测、碰撞分析、刚体动力学模拟、弹性体（包括大尺度变形）模拟、流体动力学模拟等。
- 引擎处理逻辑（右图）



碰撞检测

- 碰撞检测的任务是实时、准确地检测动态场景中发生碰撞接触的物体，为碰撞分析作准备。
- 碰撞检测是对物理引擎的仿真模拟效率起决定性作用的一个模块。因此如何用最小时空代价剔除不发生碰撞的物体对，是提高物理引擎效率的关键。

碰撞分析

- 碰撞分析以计算几何为基础，计算碰撞检测所得到的碰撞物体或准碰撞物体间的接触点信息，包括相交区域、分离距离/刺穿深度、碰撞方向以及接触时间等信息。
- 考虑到计算效率，为了加速运算，一般都是根据碰撞物体的种类和类型选择相应的简单碰撞体代替原物体参与碰撞分析。通常采用的碰撞体有：立方体、圆柱体、球体等基本形状或者原物体简化后的凸体。任意形状之间碰撞分析算法因为缺乏可用的先验特征通常都不稳定，而且效率很低，实际应用中很少采用。

动力学模拟

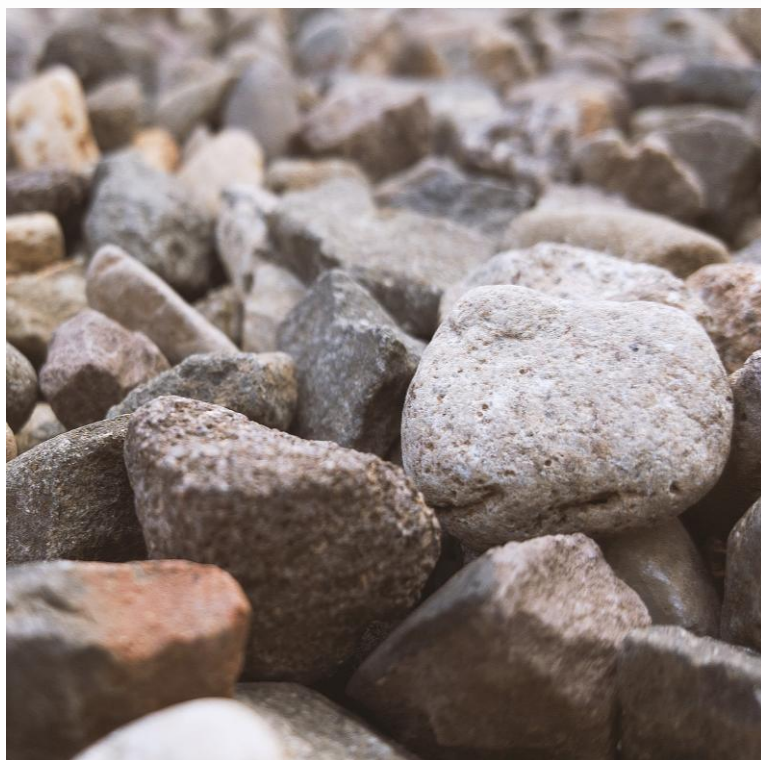
- 图形学中的动力学模拟与其他领域的动力学模拟一样，都遵循欧拉-拉格朗日运动方程，但是又有所区别：图形学的研究重点在于如何对运动方程进行简化，在精度和效率之间权衡，满足动态图形交互的需要。
 - 针对刚性物体，一般采用基于牛顿力学系统的分析方法，通过在接触点上建立约束，划归为线性互补问题，对方程组求解得到每个刚性体不发生穿刺的合理状态。
 - 对于可变形物体的模拟一般以弹/塑性力学为基础，采用有限元、有限体、以及弹簧质点模型为手段进行求解。

本课程介绍的技术点

1. 刚体模拟
2. 碰撞检测

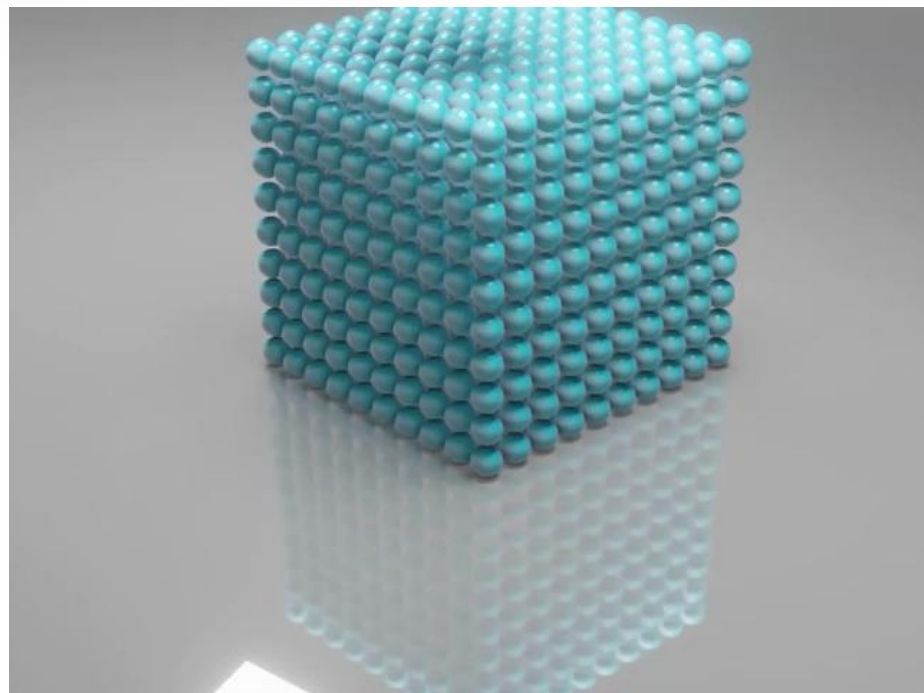
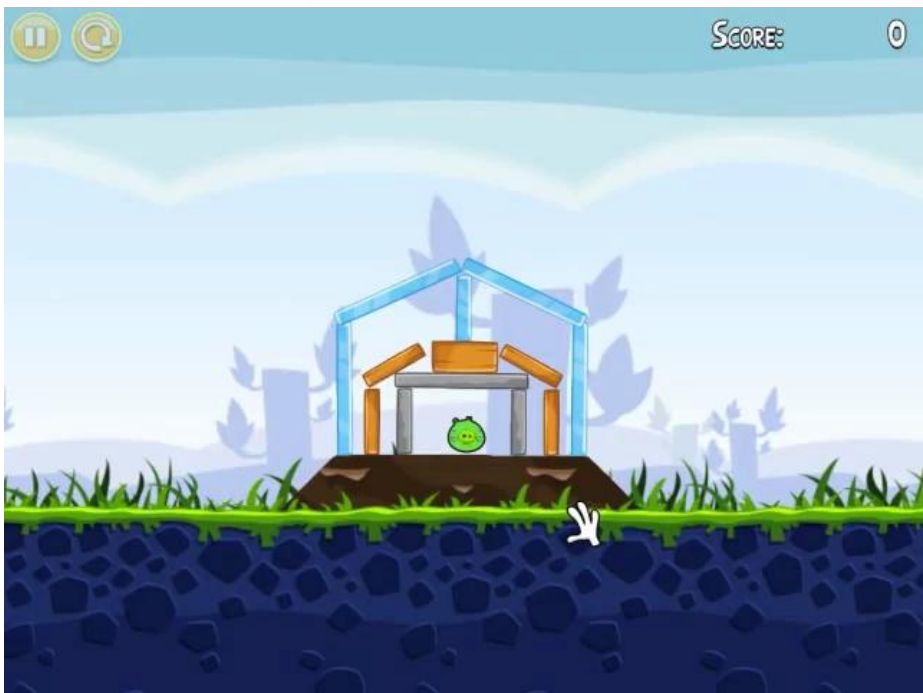
刚体模拟

- 现实世界的刚体



刚体模拟

- 虚拟世界的刚体

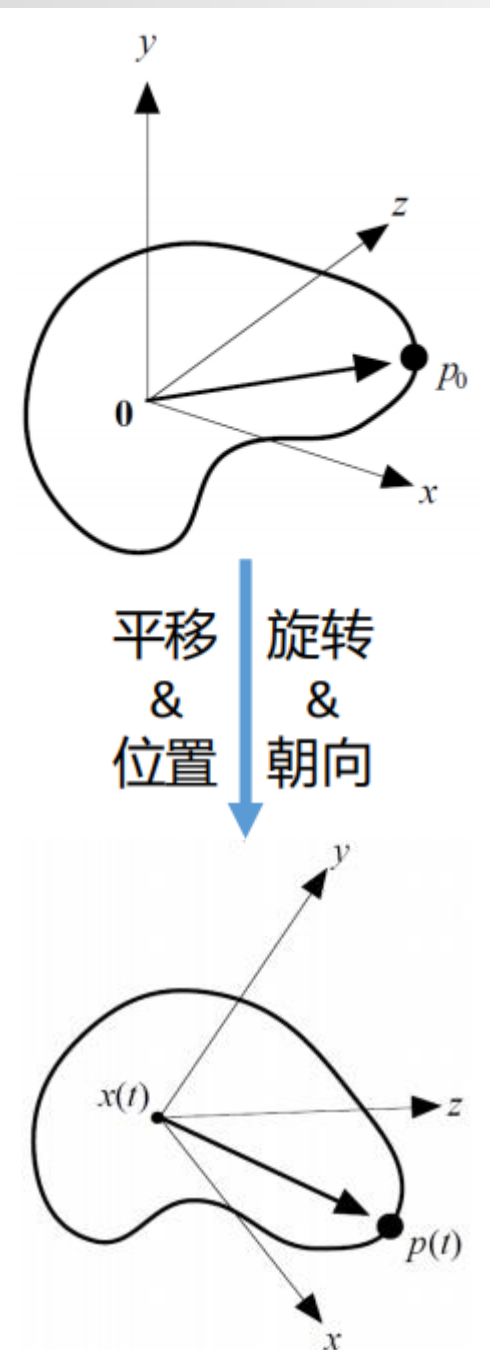


刚体运动状态

- 刚体平动状态量
 - **位置**、速度、加速度
- 刚体转动状态量
 - **朝向**、角速度

刚体模拟就是根据物理定律求解对应时间下刚体的状态量：

- 中心位置
- 朝向



刚体运动状态

平移 Translation

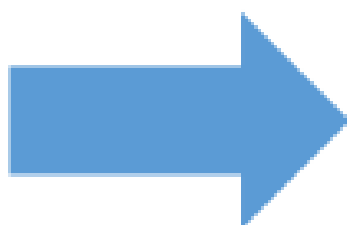
位置 Position

线速度 Linear velocity

质量 Mass

动量 Linear momentum

力 Force



旋转 Rotation

朝向 Orientation

角速度 Angular velocity

惯性张量 Inertia tensor

角动量 Angular momentum

力矩 Torque

刚体模拟

1. 状态量的表示
2. 动量定理与角动量定理
3. 刚体模拟求解

状态量的表示 - 位置与朝向

- 刚体的位置向量：用三维向量表示，表示t时刻刚体重心位置

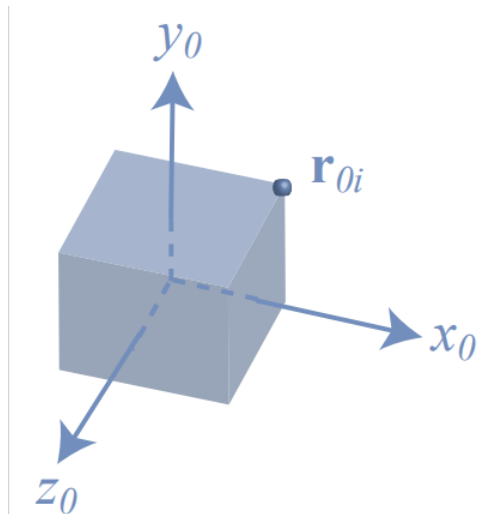
$$\mathbf{x}(t) = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- 刚体的朝向：用旋转矩阵表示，表示t时刻刚体的朝向，刚体系坐标轴在世界系中的向量

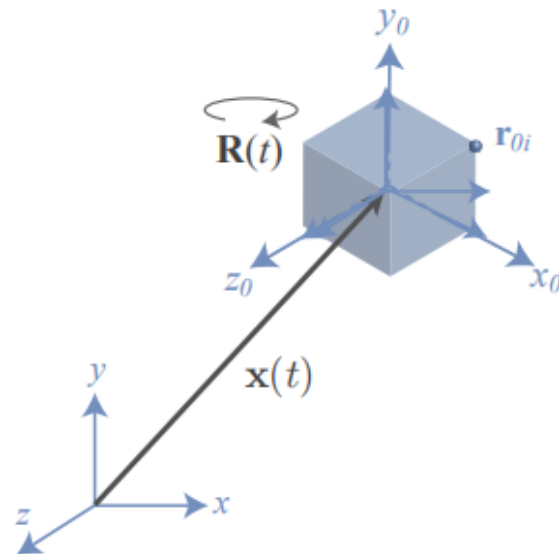
$$\mathbf{R}(t) = \begin{pmatrix} r_{xx} & r_{yx} & r_{zx} \\ r_{xy} & r_{yy} & r_{zy} \\ r_{xz} & r_{yz} & r_{zz} \end{pmatrix}$$

状态量的表示 - 刚体坐标系

- 刚体坐标系 (body space) : 一旦刚体形状定义好, 刚体上的每个位置在刚体坐标系内保持不变。
- 刚体坐标系以刚体的重心作为原点



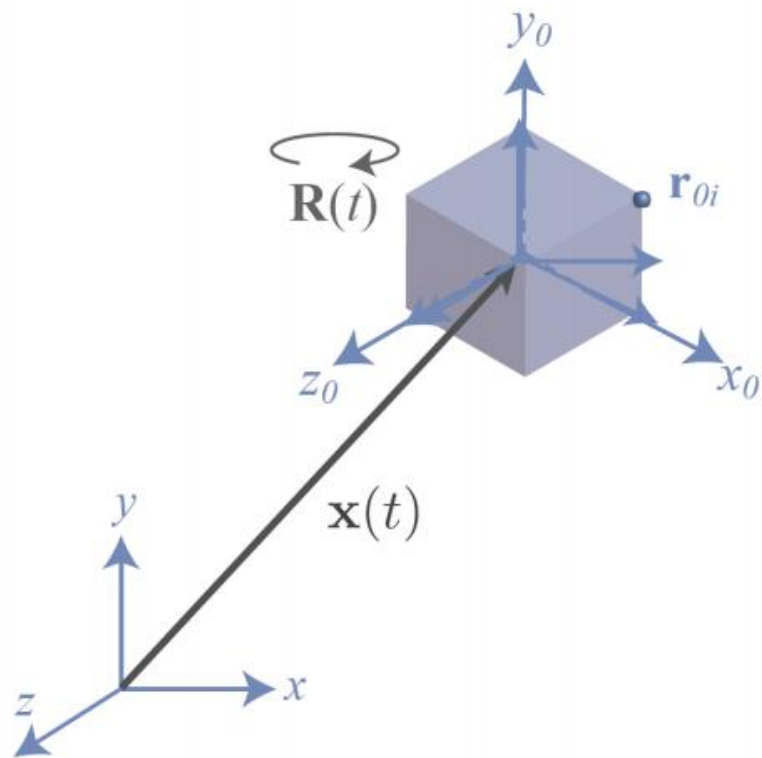
刚体坐标系



世界坐标系

状态量的表示 - 坐标系转换

- 使用 $\mathbf{x}(t)$ 与 $\mathbf{R}(t)$ 能够将刚体坐标系中的坐标转换到世界坐标系
- 对于刚体系中坐标为 $\mathbf{r}_{0i}$ 的点
- 在世界系中的坐标为
 - $\mathbf{r}_i(t) = \mathbf{x}(t) + \mathbf{R}(t)\mathbf{r}_{0i}$



状态量的表示 - 物理意义

$\mathbf{x}(t)$ 的物理意义：重心在时间 t 上的位置

$$\mathbf{R}(t) = \begin{pmatrix} r_{xx} & r_{yx} & r_{zx} \\ r_{xy} & r_{yy} & r_{zy} \\ r_{xz} & r_{yz} & r_{zz} \end{pmatrix}$$

$\mathbf{R}(t)$ 的物理意义：

- 考虑刚体坐标系中的向量 $\mathbf{x} = [1, 0, 0]^T$ ，对应世界系向量 $\mathbf{x}' = \mathbf{R}(t)\mathbf{x} = [r_{xx}, r_{xy}, r_{xz}]^T$ ，为 $\mathbf{R}(t)$ 的第一列
- 同理，刚体系中的 y, z 轴对应的世界系向量，为 $\mathbf{R}(t)$ 的第二三列
- $\mathbf{R}(t)$ 表示刚体系坐标轴在世界系中的向量

状态量的表示 - 线速度

- 我们需要知道位置 $x(t)$ 与朝向 $R(t)$ 的导数
- 位置 $x(t)$ 的导数定义为线速度 $v(t)$: 表示刚体重心在世界坐标系中的速度

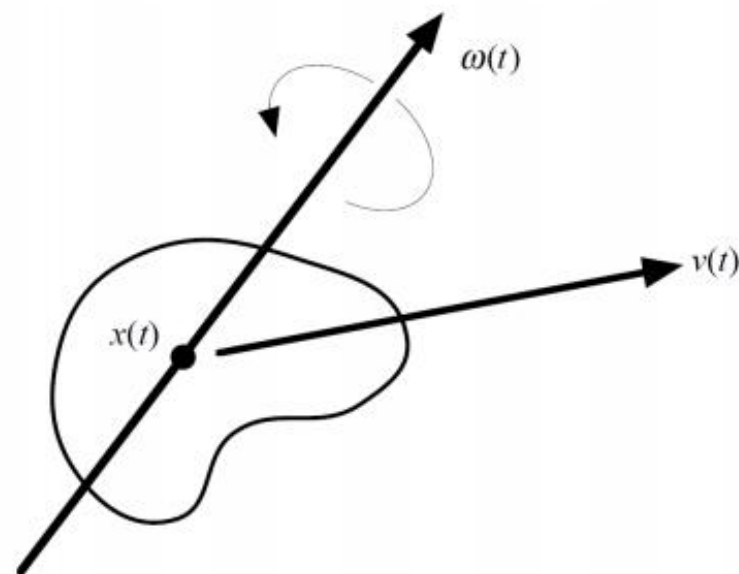
$$\dot{x}(t) = v(t)$$

状态量的表示 - 角速度

如果我们让刚体的重心静止，那么剩下的运动就只有围绕重心的转动，因此刚体运动可以分解为平移和转动

我们把转动用角速度 $\omega(t)$ 表示

- 方向：由右手螺旋定则确定
- 大小：转动的角速度，rad/s

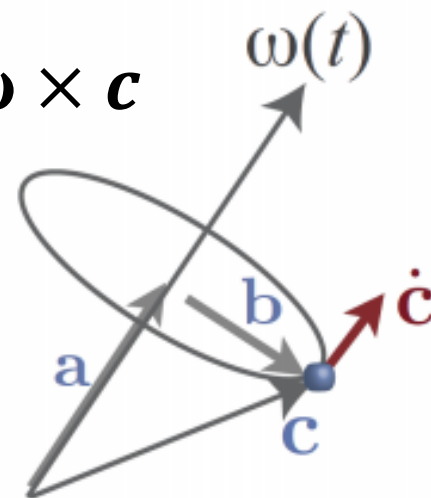


状态量的表示 - 角速度与朝向

- 仿照位置和速度的关系： $\dot{\mathbf{x}}(t) = \mathbf{v}(t)$
- 我们希望知道角速度 $\boldsymbol{\omega}(t)$ 与朝向 $\mathbf{R}(t)$ 的关系
- $\mathbf{R}(t)$ 由刚体系 xyz 轴向量组成，那么 $\dot{\mathbf{R}}(t)$ 的每一列应该表示了 xyz 轴变化的速度.

旋转的向量的导数

- 考虑世界系中的向量 c ，其起点固定在刚体上， $\dot{c}$ 表示终点的线速度
- 将 c 分解为平行和垂直于 $\omega(t)$ 的向量 a, b
- 对于半径 $|b|$ ，角速度 ω 的圆周运动，线速度 $|\dot{c}| = |\omega||b|$ ， $\dot{c}$ 的方向与 $\omega \times b$ 相同
- 故 $\dot{c} = \omega \times b = \omega \times b + \omega \times a = \omega \times c$



$R(t)$ 的导数

- 有了以上结论，就可以容易求出 $R(t)$ 的导数
- 对于 $R(t)$ 的某一系列 r （为刚体坐标系 xyz 轴之一），该列的导数为

$$\dot{r} = \omega \times r$$

- 因此有

$$\dot{R} = \left(\omega \times \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix} \quad \omega \times \begin{pmatrix} r_{yx} \\ r_{yy} \\ r_{yz} \end{pmatrix} \quad \omega \times \begin{pmatrix} r_{zx} \\ r_{zy} \\ r_{zz} \end{pmatrix} \right)$$

叉乘的矩阵表示

- 由于以上表达式略显冗余，我们可以做如下简化：
考虑向量 a, b 的叉乘

$$a \times b = \begin{pmatrix} -a_z b_y + a_y b_z \\ a_z b_x - a_x b_z \\ -a_y b_x + a_x b_y \end{pmatrix}$$

- 令 $a^* =$

$$\begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}$$

- 叉乘可以表示为矩阵 $\times$ 向量

$$- a \times b = a^* b$$

$R(t)$ 的导数

- 于是有

$$\begin{aligned}\dot{\mathbf{R}}(t) &= \begin{pmatrix} \omega^* \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix} & \omega^* \begin{pmatrix} r_{yx} \\ r_{yy} \\ r_{yz} \end{pmatrix} & \omega^* \begin{pmatrix} r_{zx} \\ r_{zy} \\ r_{zz} \end{pmatrix} \end{pmatrix} \\ &= \boldsymbol{\omega}(t)^* \mathbf{R}(t)\end{aligned}$$

- 向量导数: $\dot{\mathbf{c}} = \boldsymbol{\omega} \times \mathbf{c}$
- 矩阵导数: $\dot{\mathbf{R}}(t) = \boldsymbol{\omega}(t)^* \mathbf{R}(t)$

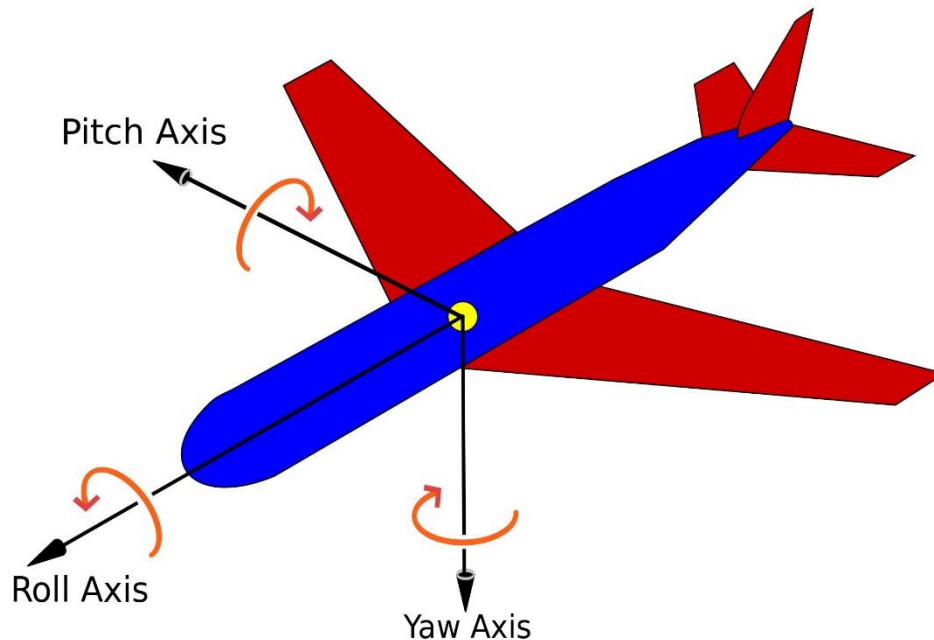
刚体动力学 - 旋转运动

- 旋转的表示方法 – 旋转矩阵
- 表示冗余：9个变量，3自由度
- 不直观
- 难以定义关于时间的导数(旋转速度)

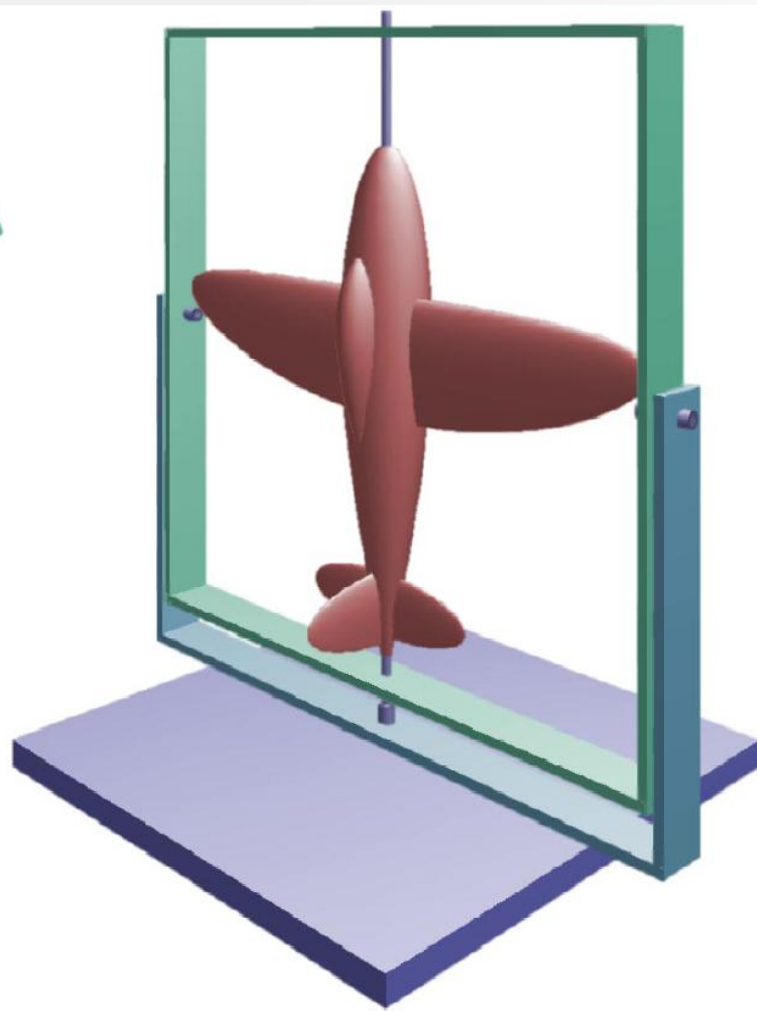
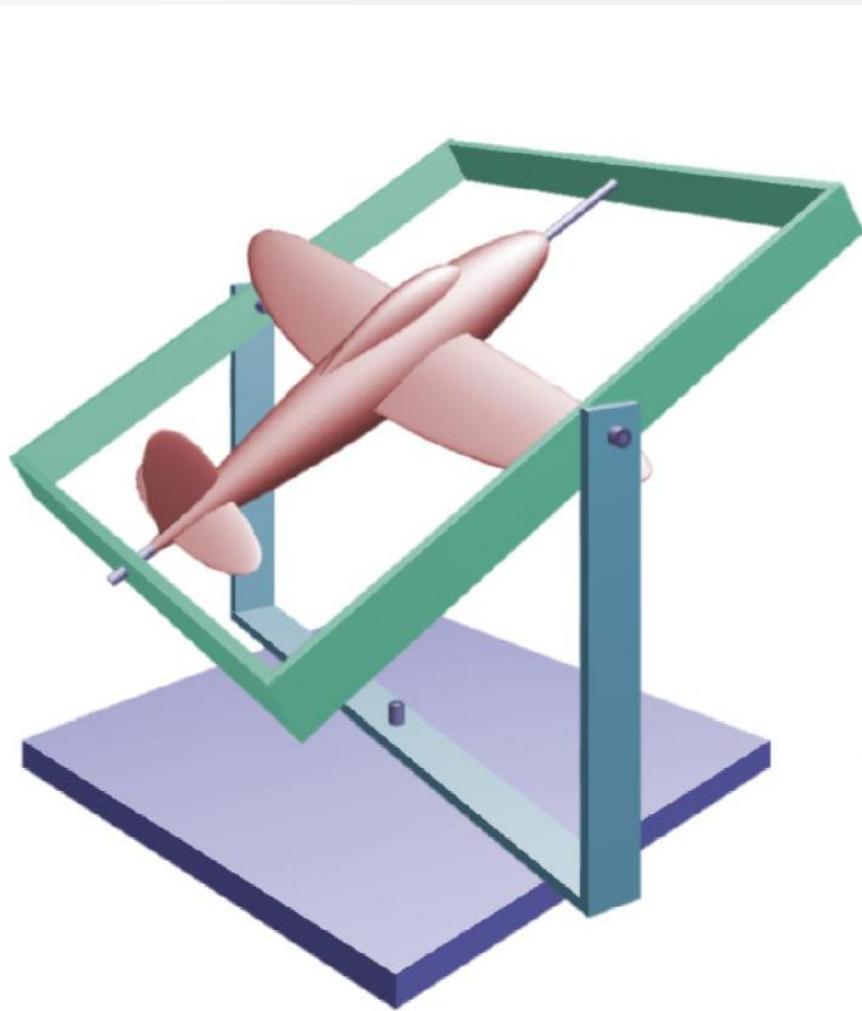
$$\mathbf{R} = \begin{bmatrix} r_{00} & r_{01} & r_{02} \\ r_{10} & r_{11} & r_{12} \\ r_{20} & r_{21} & r_{22} \end{bmatrix}$$

刚体动力学-旋转运动

- 旋转的表示方法 – 欧拉角
- 用三个绕轴旋转表示任意旋转，直观
- 但在某些情况下会出现万向锁，失去自由度

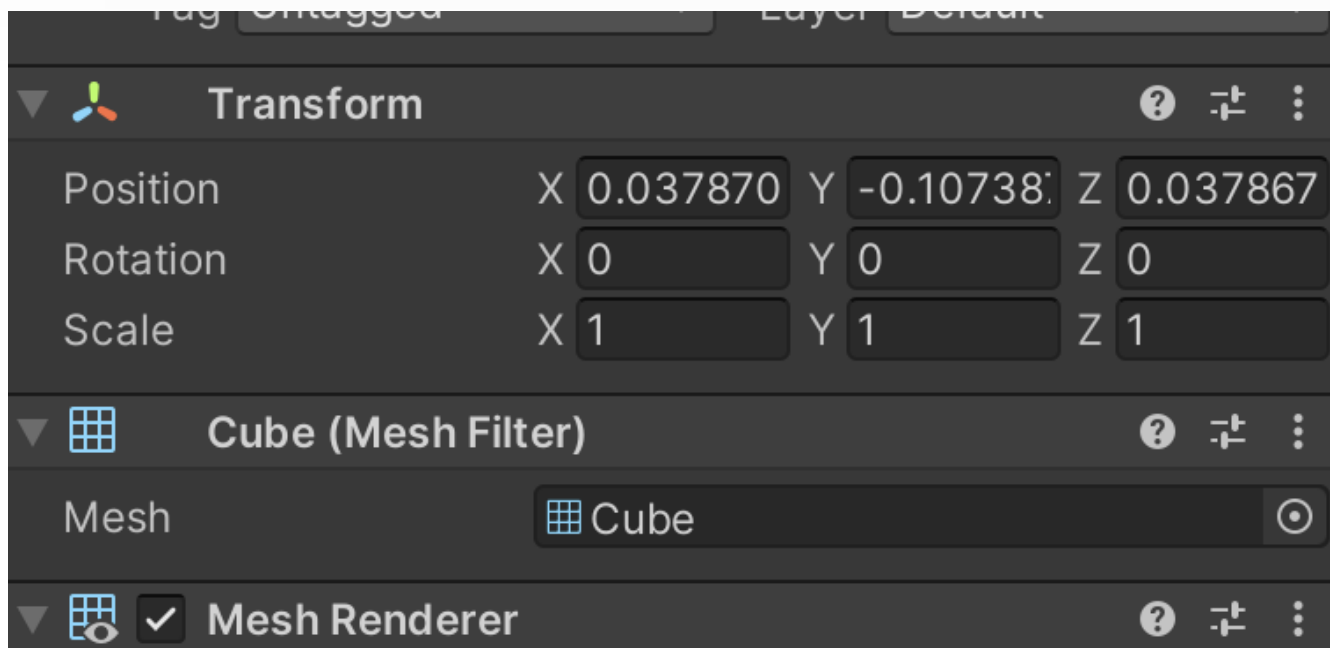


刚体动力学 - 万向锁



刚体动力学 - 欧拉角

- Unity: 按照Z-X-Y的旋转顺序



状态量的表示 - 四元数

**Complex
multiplications**

	1	i
1	1	i
i	i	-1

复平面的数与二维空间对应

**Quaternion
multiplications**

	1	i	j	k
1	1	i	j	k
i	i	-1	k	-j
j	j	-k	-1	i
k	k	j	-i	-1

四元数与三维空间对应

如果仅仅使用三维向量表示，只能定义加减两种运算，但是不能定义乘除

四元数及其运算

- 定义

$$\mathbf{q} = w + xi + yj + zk \text{ 或写成 } \mathbf{q} = s + \mathbf{v}$$

- 标量乘 $a\mathbf{q} = [as \quad a\mathbf{v}]$

- 加减 $\mathbf{q}_1 \pm \mathbf{q}_2 = [s_1 \pm s_2 \quad \mathbf{v}_1 \pm \mathbf{v}_2]$

- 乘 $\mathbf{q}_1 \times \mathbf{q}_2 = [s_1s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \quad s_1\mathbf{v}_2 + s_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2]$

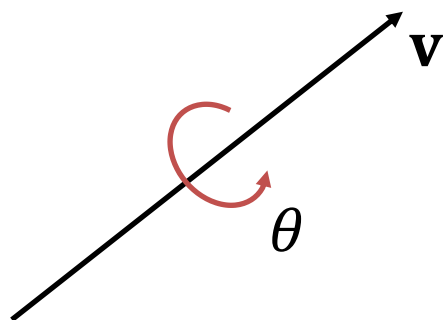
- 取模 $\|\mathbf{q}\| = \sqrt{s^2 + \mathbf{v} \cdot \mathbf{v}}$

- 共轭 $\bar{\mathbf{q}} = s - \mathbf{v}$

- 逆 $\mathbf{q}^{-1} = \frac{\bar{\mathbf{q}}}{\|\mathbf{q}\|}$

状态量的表示 – 四元数

- 用单位四元数表示旋转： $s = \cos \frac{\theta}{2}$, $\mathbf{v}$ 与旋转轴同向



$$\begin{cases} \mathbf{q} = [\cos \frac{\theta}{2} & \mathbf{v}] \\ \|\mathbf{q}\| = 1 \end{cases}$$



$$\begin{cases} \mathbf{q} = [\cos \frac{\theta}{2} & \mathbf{v}] \\ \|\mathbf{v}\|^2 = \sin^2 \frac{\theta}{2} \end{cases}$$

- 优势：计算方便，表示简洁，直观

状态量的表示 – 四元数与旋转矩阵

- 对 P 点进行旋转，将 P 表示为实部为0的四元数：

$$\phi_q(P) = \mathbf{q}P\mathbf{q}^{-1}$$

- 可以验证与罗德里格斯旋转公式相同
- 对应旋转矩阵

$$\mathbf{R} = \begin{bmatrix} s^2 + x^2 - y^2 - z^2 & 2(xy - sz) & 2(xz + sy) \\ 2(xy + sz) & s^2 - x^2 + y^2 - z^2 & 2(yz - sx) \\ 2(xz - sy) & 2(yz + sx) & s^2 - x^2 - y^2 + z^2 \end{bmatrix}$$

状态量的表示 – 四元数的导数

- 在 Δt 时间内，四元数 q 的变化量为：

$$\Delta q \approx \left[\cos \left(\frac{\|\omega\| \Delta t}{2} \right), \frac{\omega}{\|\omega\|} \sin \left(\frac{\|\omega\| \Delta t}{2} \right) \right]$$

- 对于无穷小时间 Δt , $\cos(\theta) = 1$, $\sin(\theta) = \theta$:

$$\Delta q = [1, \frac{1}{2}\omega \Delta t]$$

- 在世界坐标系下施加微小旋转，等效于先做该旋转，再做原来的旋转：

$$q(t + \Delta t) = \Delta q \otimes q(t)$$

状态量的表示 – 四元数的导数

- 接上页:

$$q(t + \Delta t) - q(t) = \Delta q \otimes q(t) - q(t) = (\Delta q - 1) \otimes q(t)$$

$$\frac{q(t + \Delta t) - q(t)}{\Delta t} = \frac{\Delta q - 1}{\Delta t} \otimes q(t)$$

- 因此对 Δt , 取极限 $\Delta t \rightarrow 0$:

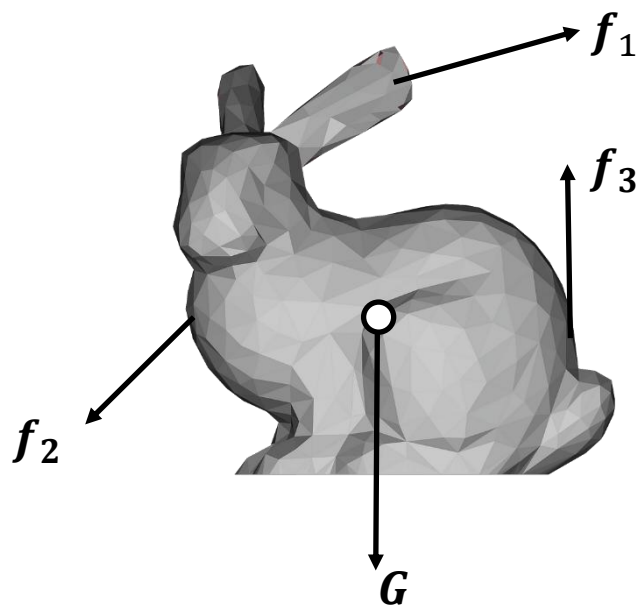
$$\dot{q} = \frac{1}{2} \begin{bmatrix} 0 \\ \boldsymbol{\omega}_w \end{bmatrix} \otimes q$$

刚体模拟

1. 状态量的表示
2. 动量定理与角动量定理
3. 刚体模拟求解

动量定理

- 物体在一段时间内所受合外力的冲量，等于它在这段时间内动量的变化量。



动量定理微分形式：

$$d\mathbf{J} = d\mathbf{p}$$

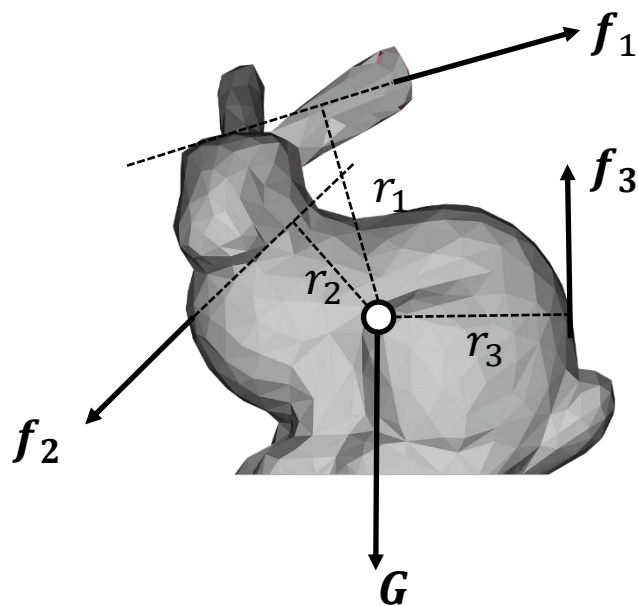
$$\mathbf{F} dt = d\mathbf{p} = d(m\mathbf{v}) = m d\mathbf{v}$$

可推导速度表达式：

$$d\mathbf{v} = \frac{\mathbf{F}}{m} dt$$

角动量定理

- **惯性系下**，物体（或系统）对某固定参考点所受的合外力矩，等于它对该点的角动量随时间的变化率。



角动量定理微分形式：

$$\tau dt = d\mathbf{L}$$

其中

$$\tau = \sum \mathbf{r}_i \times \mathbf{f}_i \text{ 是合力矩}$$

在质心系（以质心为原点，坐标轴与世界坐标轴平行）下使用角动量定理

角动量 L 的微分

在质心系（以质心为原点，坐标轴与世界坐标轴平行）下的角动量完全由转动提供

$$\mathbf{L}_C = \sum_i m_i \mathbf{r}_i \times (\boldsymbol{\omega} \times \mathbf{r}_i)$$
$$\mathbf{L}_C = \sum_i m_i [(\mathbf{r}_i \cdot \mathbf{r}_i) \boldsymbol{\omega} - (\mathbf{r}_i \cdot \boldsymbol{\omega}) \mathbf{r}_i]$$

计算 x 分量

$$\begin{aligned} L_x &= \sum_i m_i [(x_i^2 + y_i^2 + z_i^2) \omega_x - (x_i \omega_x + y_i \omega_y + z_i \omega_z) x_i] \\ &= \sum_i m_i [(y_i^2 + z_i^2) \omega_x - x_i y_i \omega_y - x_i z_i \omega_z] \\ &= \left[\sum_i m_i (y_i^2 + z_i^2) \right] \omega_x - \left[\sum_i m_i x_i y_i \right] \omega_y - \left[\sum_i m_i x_i z_i \right] \omega_z \end{aligned}$$

角动量 L 的微分

角动量的三个分量可以写为

$$\begin{aligned}L_x &= I_{xx}\omega_x + I_{xy}\omega_y + I_{xz}\omega_z \\L_y &= I_{yx}\omega_x + I_{yy}\omega_y + I_{yz}\omega_z \\L_z &= I_{zx}\omega_x + I_{zy}\omega_y + I_{zz}\omega_z\end{aligned}$$

写成紧凑的矩阵形式

$$\begin{bmatrix} L_x \\ L_y \\ L_z \end{bmatrix} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

$$\begin{aligned}\mathbf{I} &= \begin{pmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{pmatrix} \\ &= \sum_i \begin{pmatrix} m_i(y_i^2 + z_i^2) & -m_i x_i y_i & -m_i x_i z_i \\ -m_i x_i y_i & m_i(x_i^2 + z_i^2) & -m_i y_i z_i \\ -m_i x_i z_i & -m_i y_i z_i & m_i(x_i^2 + y_i^2) \end{pmatrix}\end{aligned}$$

这个矩阵即为**惯性张量**，是角速度到角动量的变换矩阵

$$\mathbf{L} = \mathbf{I}\boldsymbol{\omega}$$

角动量 L 的微分

L 对时间的导数可以写为：

$$\frac{d\mathbf{L}}{dt} = \frac{d(\mathbf{I}\boldsymbol{\omega})}{dt} = \dot{\mathbf{I}}\boldsymbol{\omega} + \mathbf{I}\dot{\boldsymbol{\omega}}$$

可以推导右式为：

$$\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})$$

根据角动量定理：

$$d\boldsymbol{\omega} = \mathbf{I}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})) dt$$

惯性张量

- 上文的表达式包含随时间变化的坐标 $\mathbf{r}_i(t)$, 每一帧需要对所有粒子重算, 开销过大!
- 但是, 刚体坐标系中粒子相对坐标固定, 因此刚体坐标系的惯性张量 $\mathbf{I}_{body}$ 不变的
- 能否在 $\mathbf{I}$ 与 $\mathbf{I}_{body}$ 之间找到变换关系?

惯性张量

- 寻找 $\mathbf{I}$ 与 $\mathbf{I}_{body}$ 的联系:

$$\begin{aligned}\mathbf{I} &= \sum \begin{pmatrix} m_i(y_i^2 + z_i^2) & -m_ix_iy_i & -m_ix_iz_i \\ -m_ix_iy_i & m_i(x_i^2 + z_i^2) & -m_iy_iz_i \\ -m_ix_iz_i & -m_iy_iz_i & m_i(x_i^2 + y_i^2) \end{pmatrix} \\ &= \sum m_i \mathbf{r}_i^T \mathbf{r}_i \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \begin{pmatrix} m_ix_i^2 & m_ix_iy_i & m_ix_iz_i \\ m_ix_iy_i & m_iy_i^2 & m_iy_iz_i \\ m_ix_iz_i & m_iy_iz_i & m_iz_i^2 \end{pmatrix} \\ &= \sum m_i \left((\mathbf{r}_i^T \mathbf{r}_i) \mathbf{1} - \mathbf{r}_i \mathbf{r}_i^T \right)\end{aligned}$$

惯性张量

$$\begin{aligned}\mathbf{I} &= \sum m_i \left((\mathbf{r}_i^T \mathbf{r}_i) \mathbf{1} - \mathbf{r}_i \mathbf{r}_i^T \right) \\ &= \sum m_i \left((\mathbf{R}(t) \mathbf{r}_{0i})^T (\mathbf{R}(t) \mathbf{r}_{0i}) \mathbf{1} - (\mathbf{R}(t) \mathbf{r}_{0i}) (\mathbf{R}(t) \mathbf{r}_{0i})^T \right) \\ &= \sum m_i \left(\mathbf{R}(t) (\mathbf{r}_{0i}^T \mathbf{r}_{0i}) \mathbf{R}(t)^T \mathbf{1} - \mathbf{R}(t) \mathbf{r}_{0i} \mathbf{r}_{0i}^T \mathbf{R}(t)^T \right) \\ &= \mathbf{R}(t) \left(\sum m_i \left((\mathbf{r}_{0i}^T \mathbf{r}_{0i}) \mathbf{1} - \mathbf{r}_{0i} \mathbf{r}_{0i}^T \right) \right) \mathbf{R}(t)^T \\ &= \mathbf{R}(t) \mathbf{I}_{body} \mathbf{R}(t)^T\end{aligned}$$

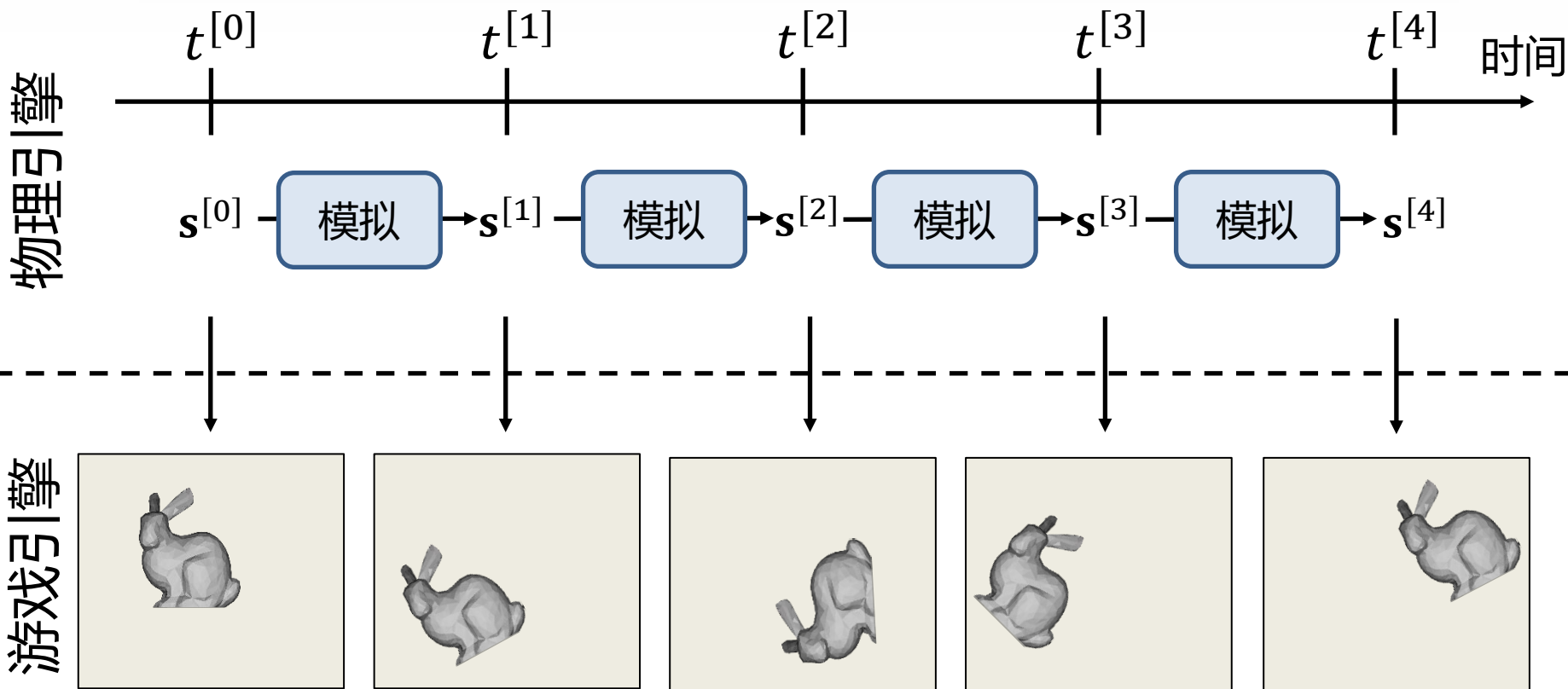
- 因此, $\mathbf{I}$ 与 $\mathbf{I}_{body}$ 可以通过旋转矩阵 $\mathbf{R}(t)$ 相联系

刚体模拟

1. 状态量的表示
2. 动量定理与角动量定理
3. 刚体模拟求解

刚体模拟求解

- 目的：随时间更新代表刚体的状态量(平移/旋转)



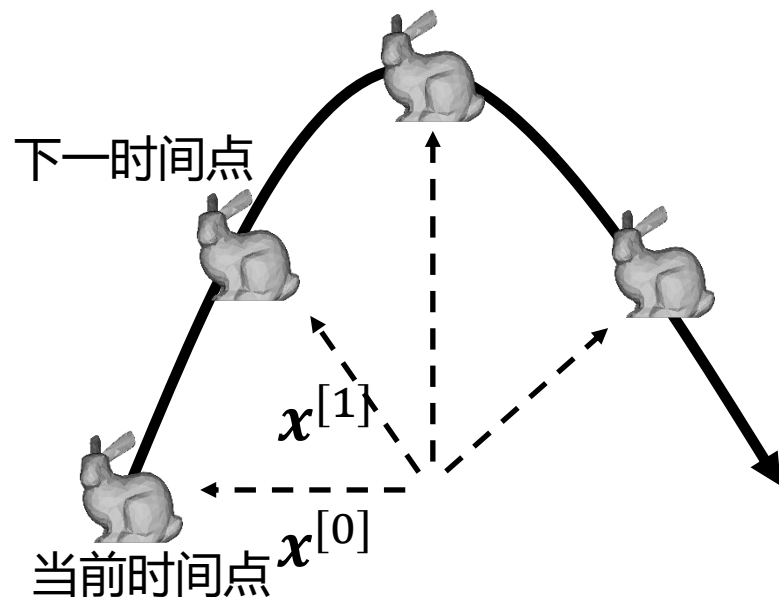
刚体模拟求解 - 平动部分

平动状态量: 位置 x , 速度 v .

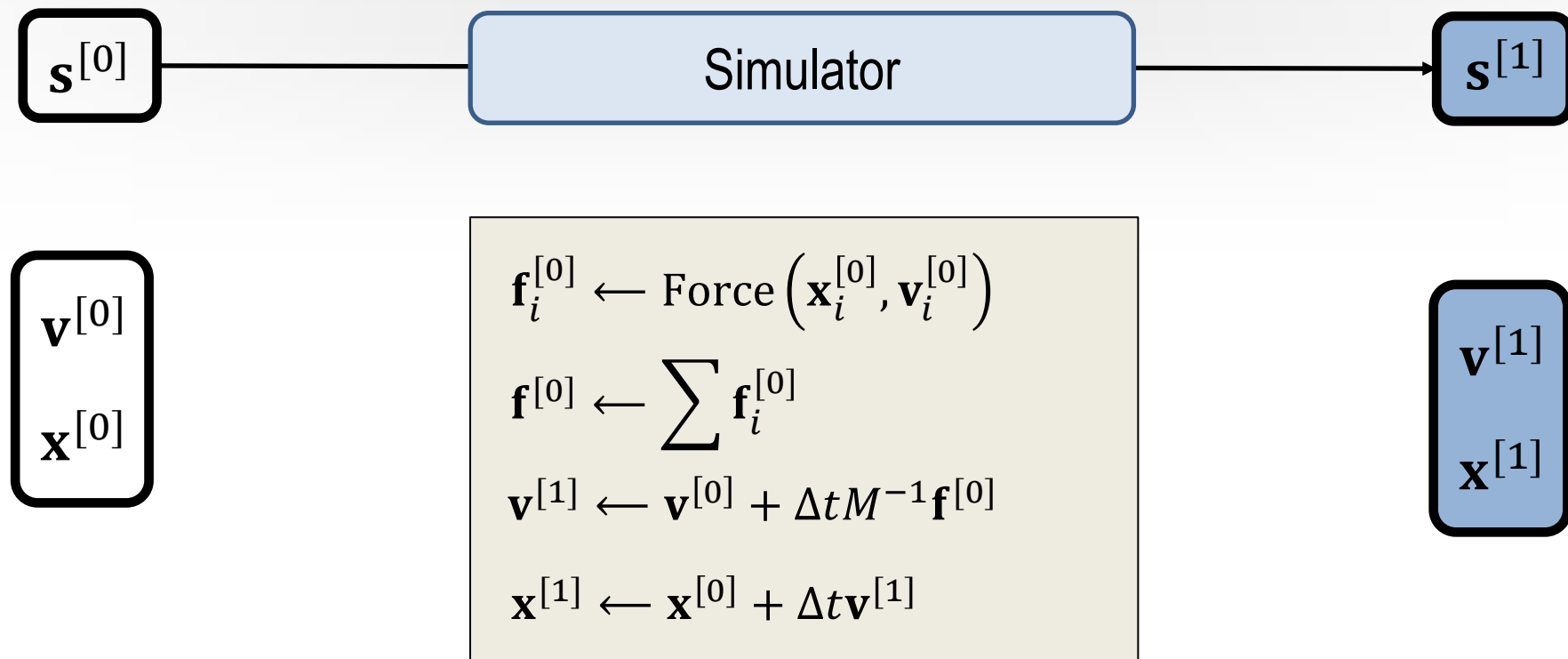
受力: f , 质量: M

可根据动量定理求解:

$$\begin{cases} v(t^{[1]}) = v(t^{[0]}) + M^{-1} f \Delta t \\ x(t^{[1]}) = x(t^{[0]}) + v(t^{[1]}) \Delta t \end{cases}$$

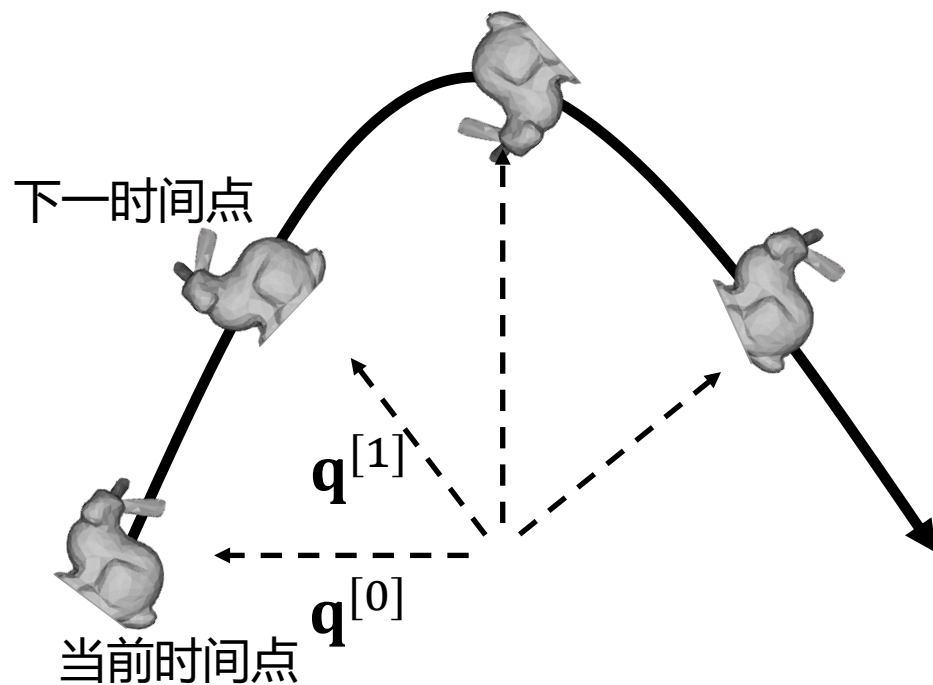
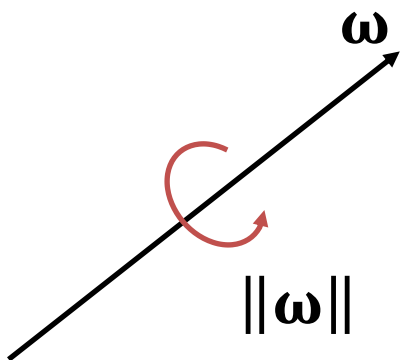


刚体模拟求解 - 平动部分



刚体动力学-旋转运动

- 旋转运动状态量：
- 角速度 ω ：方向为旋转轴，大小为旋转速度
- 四元数 $\mathbf{q}$ ：表示方向



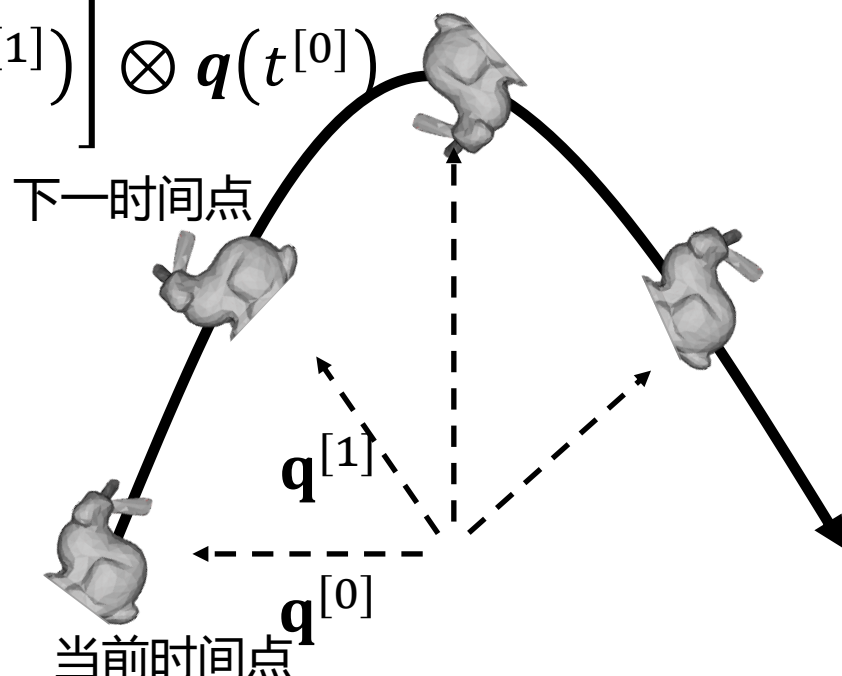
刚体动力学-旋转运动

转动状态量：朝向 q , 角速度 ω

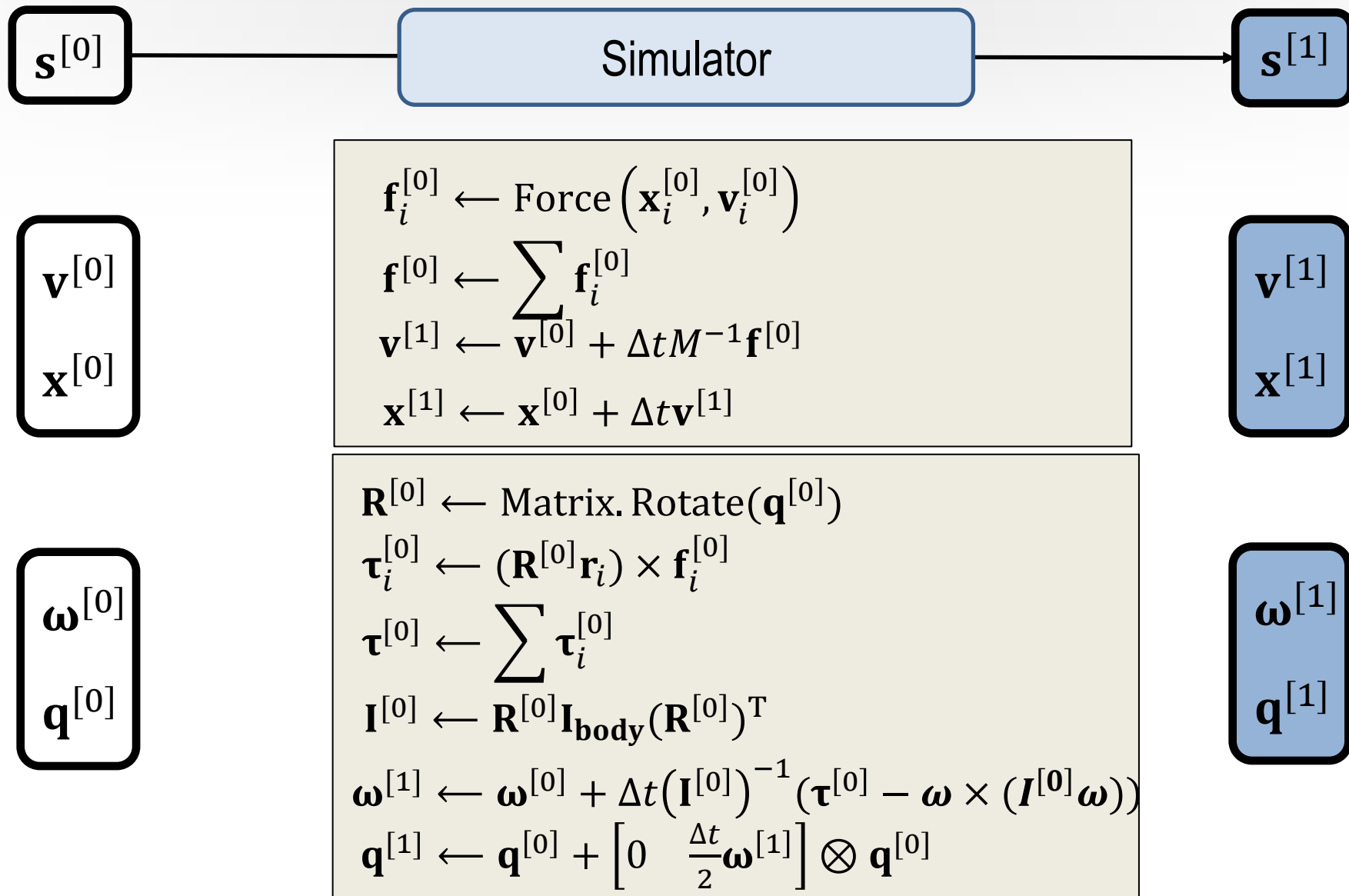
合力矩： τ , 惯性张量： I

可根据角动量定理求解：

$$\begin{cases} \omega(t^{[1]}) = \omega(t^{[0]}) + I^{-1}(\tau - \omega \times (I\omega)) \Delta t \\ q(t^{[1]}) = q(t^{[0]}) + \left[\mathbf{0} \quad \frac{\Delta t}{2} \omega(t^{[1]}) \right] \otimes q(t^{[0]}) \end{cases}$$



刚体模拟求解



下列关于刚体动力学的选项中，正确的是：

- ☒ A 刚体在某一参考系下运动时，刚体上某一点可能相对该参考系静止
- ☒ B 给出刚体任意时刻质心的位置、刚体的朝向、及某一点在刚体坐标系下的位置，即可计算该点的线/角速度/加速度
- ☒ C 一个刚体圆盘绕中心轴旋转时，其上任意一点（非中心）角速度与角动量方向相同
- ☒ D 四元数常用于表示刚体的旋转

提交

本课程介绍的技术点

1. 刚体模拟
2. 碰撞检测

碰撞检测

- 虚拟现实不可回避的问题之一
- 重要应用：
 - 虚拟现实中的三维选择和拾取
 - 力觉触觉计算：需要 $>1\text{kHz}$
 - 场景漫游中避免穿越障碍物
 - 角色之间互动
 - 军事中的击中和破坏仿真
 - 虚拟装配中避免冲突
 -

碰撞检测

- 基本任务
 - 判断空间中多个几何体模型在一定的时间段内是否相交的一个布尔论断问题
- 确定两个或多个物体彼此间是否发生接触或穿透。
 - 确定物体间发生碰撞的时间和部位
 - 虚拟现实的实时性要求决定了碰撞检测算法也必须具备实时性

碰撞检测

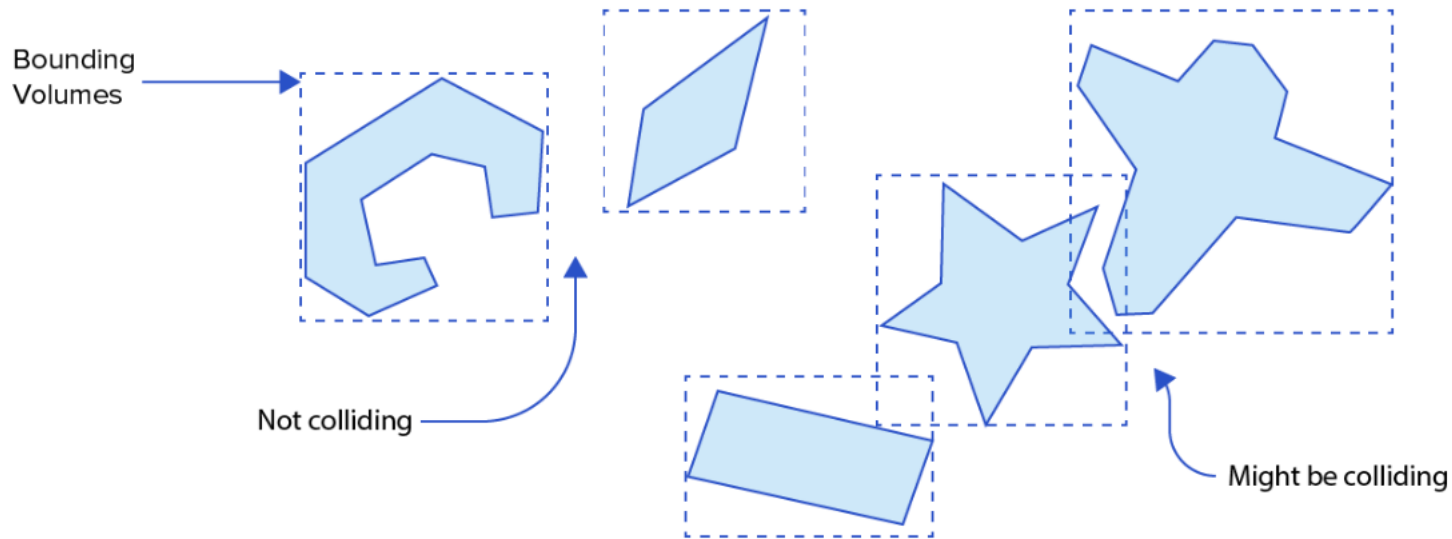
- 难点
 - 巨大的计算复杂度
 - 静态：虚拟场景中的物体模型越来越复杂，如两个分别包含 n 、 m 多边形的物体，碰撞检测复杂度 $n * m$
 - 需在很多物体之间进行碰撞检测
 - 在仿真中对实时性和真实性的要求越来越高

碰撞检测

- 物体数量 N : 计算复杂度 $O(N^2)$, 需要加速
- 分为两阶段:
 - Broad phase: 快速筛选可能的碰撞对
 - Narrow phase: 对可能的碰撞对进行碰撞检测

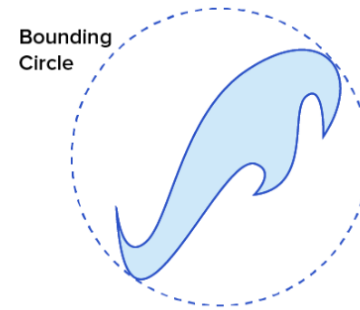
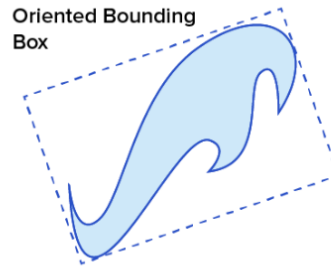
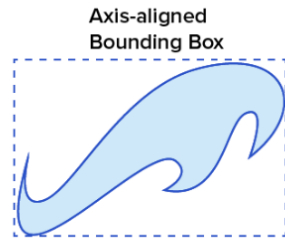
碰撞检测 – Broad phase

- 空间划分 – 包围体
- 如果包围体不相交，那么原物体也不会相交



碰撞检测 – Broad phase

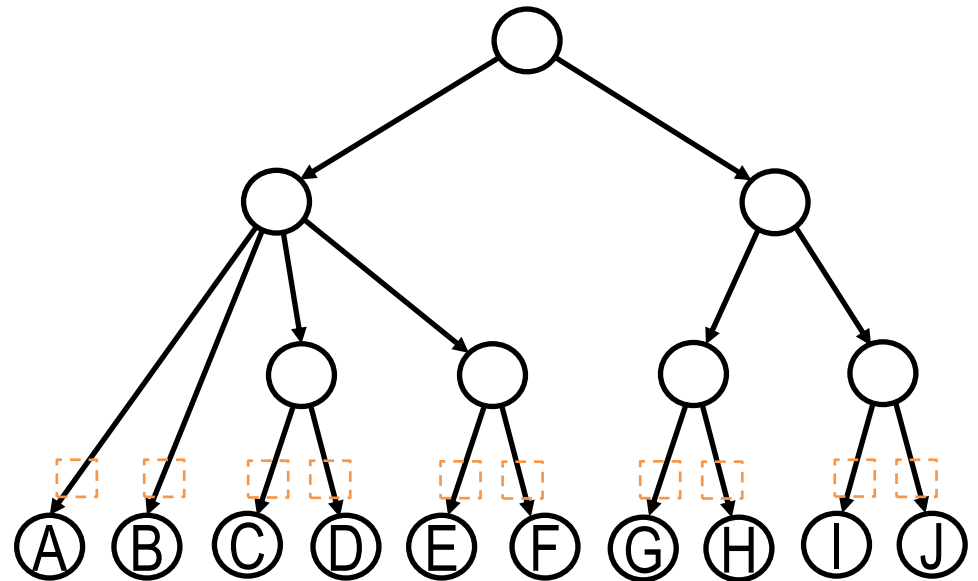
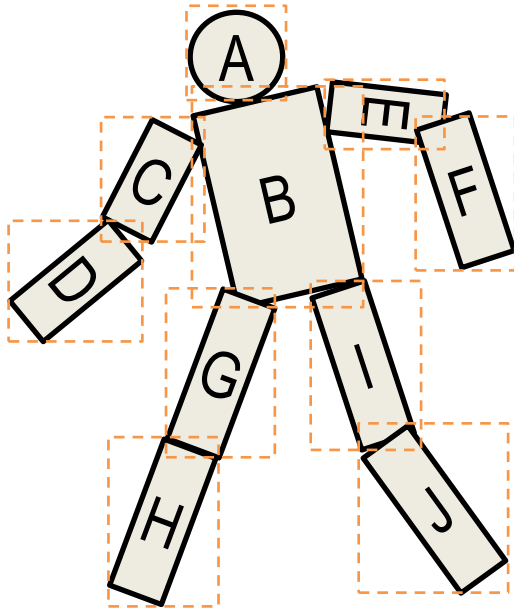
- 几种不同的包围体



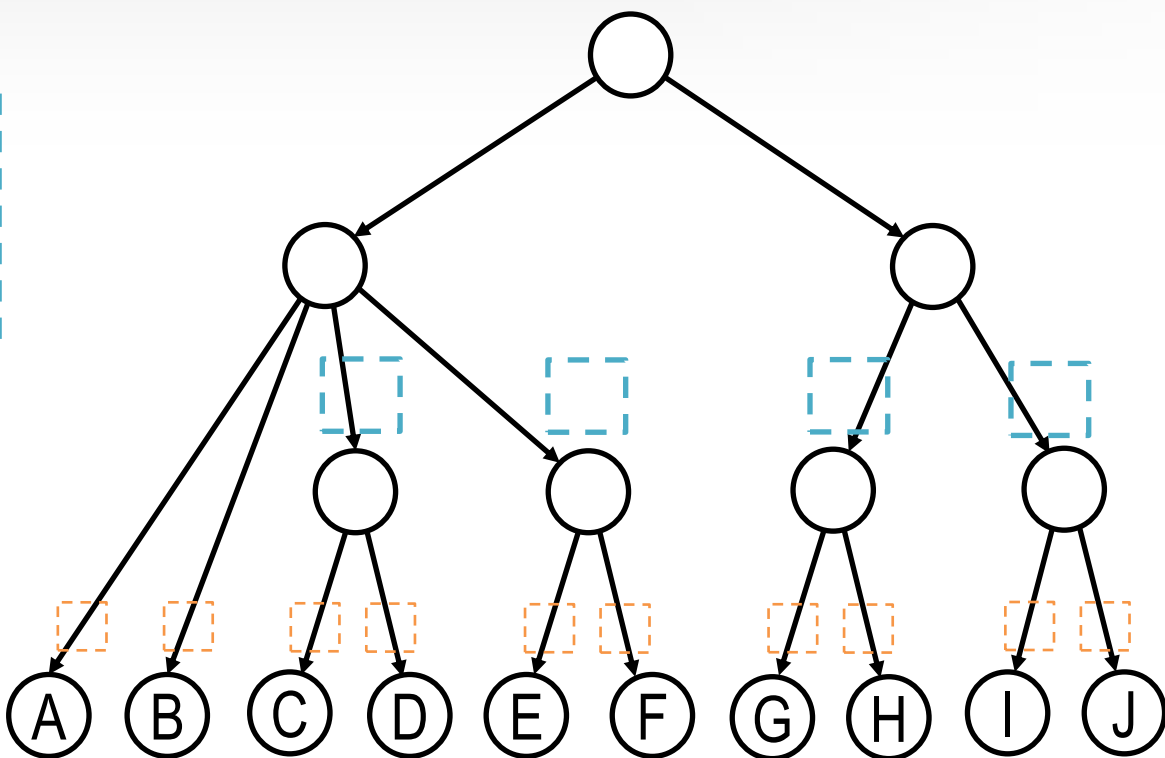
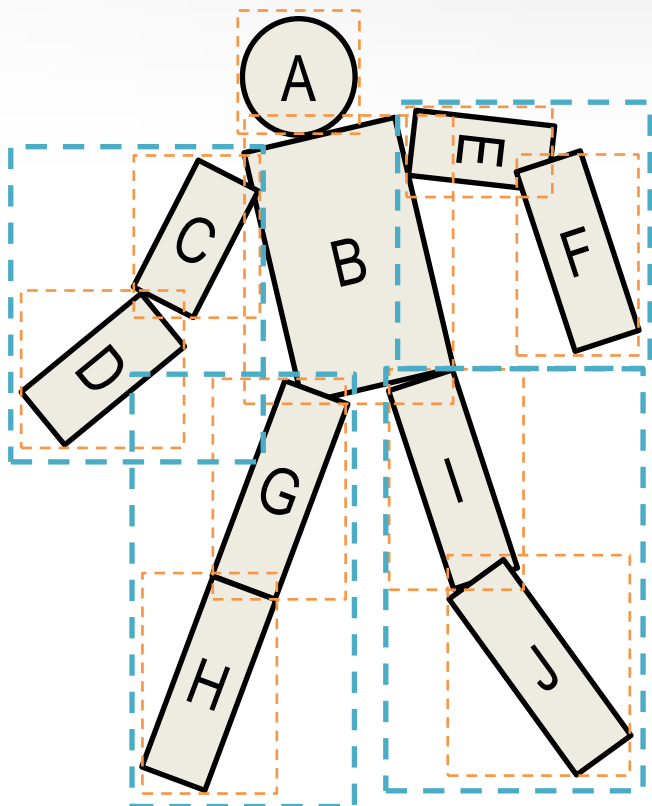
- AABB：轴平行，相交检测快(判断各个轴上的投影是否都相交)
- OBB：空间利用率高，产生更少的潜在碰撞对
- 包围球：相交检测简单，构造相对复杂

碰撞检测 – Broad phase

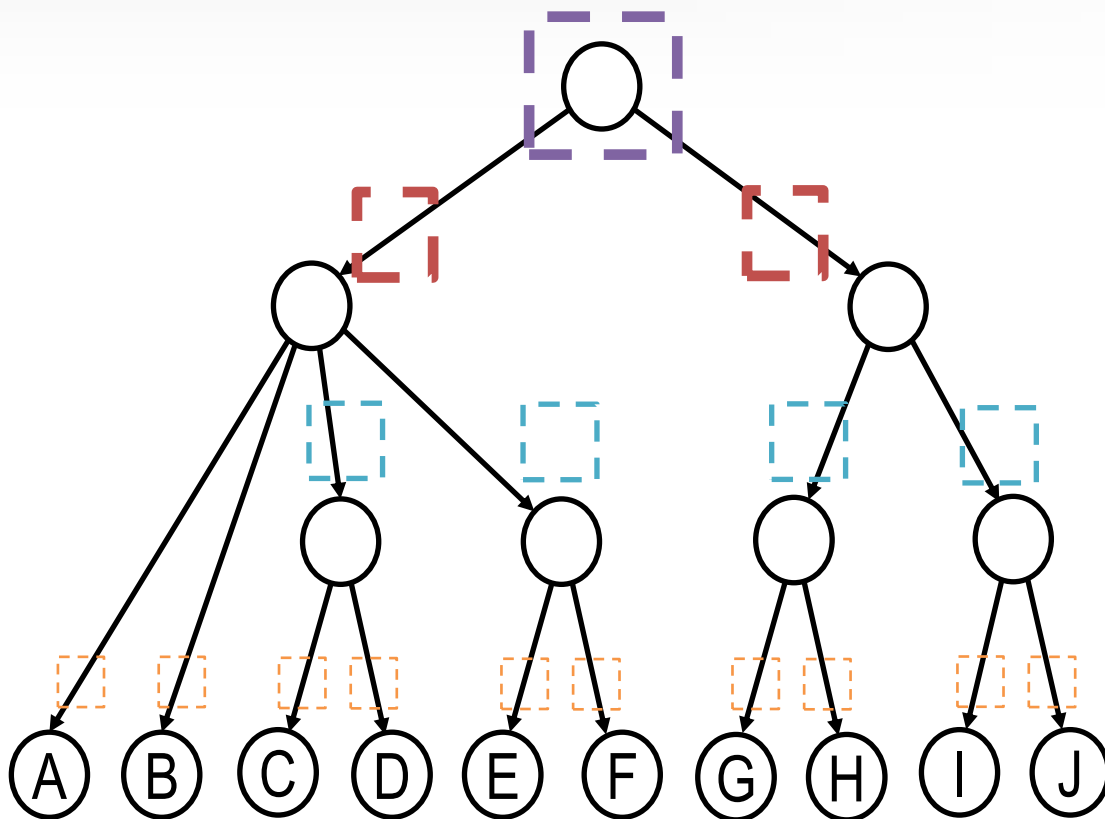
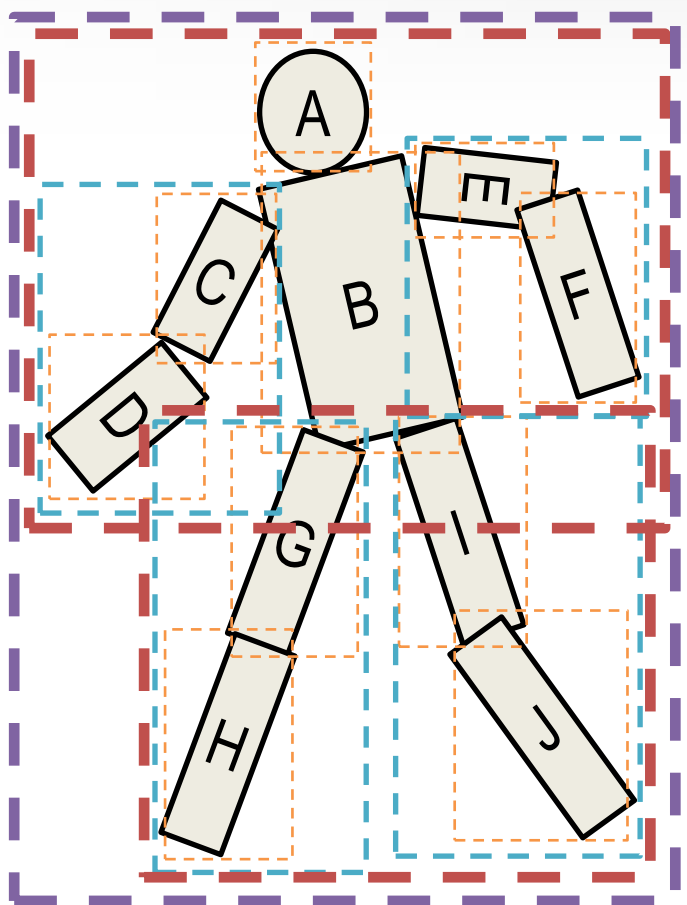
- 空间划分 – 层次包围体
- 将包围体用树形方式组合



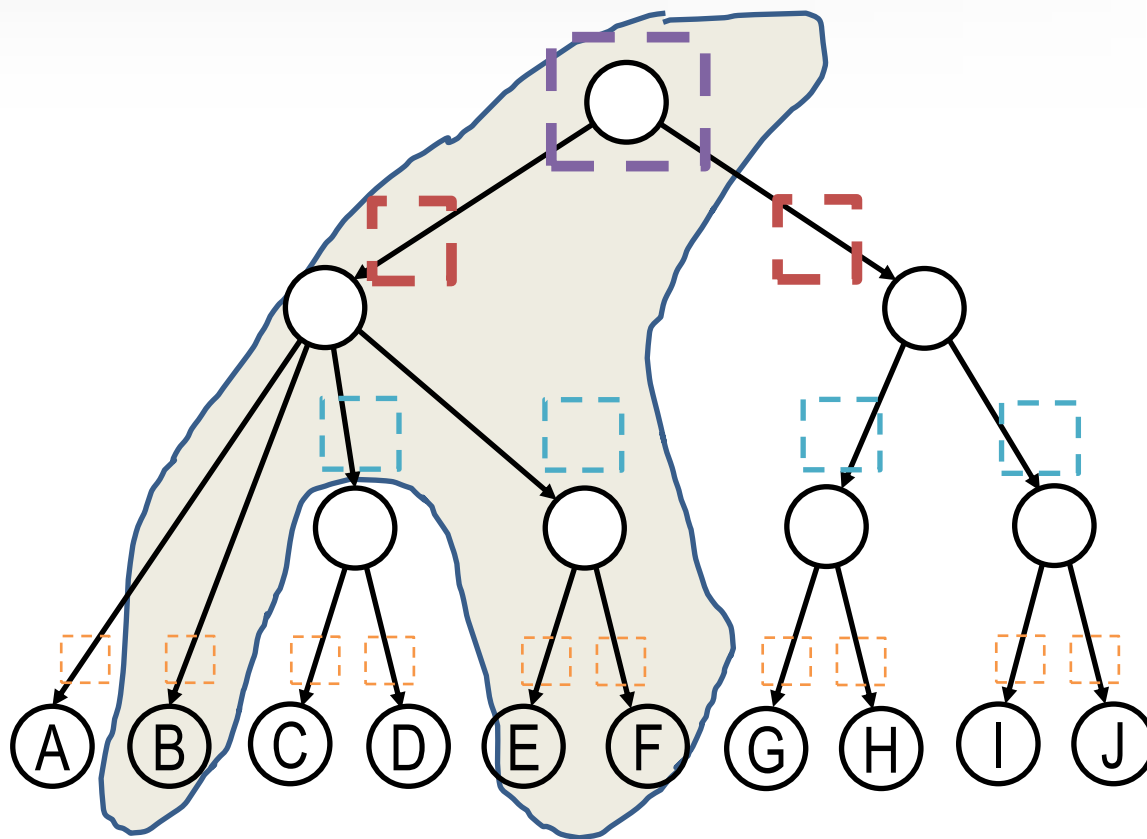
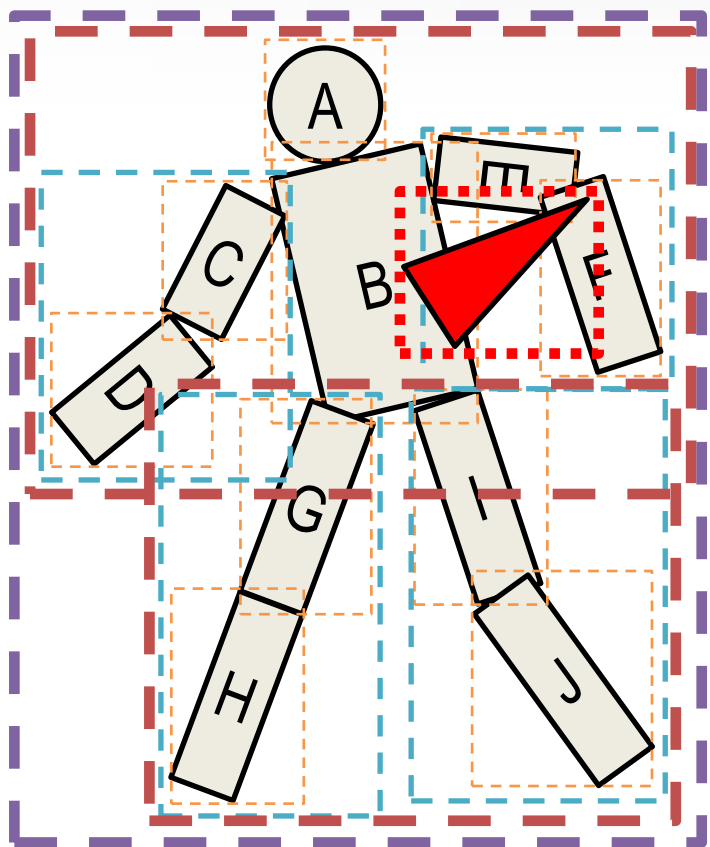
碰撞检测 – Broad phase



碰撞检测 – Broad phase

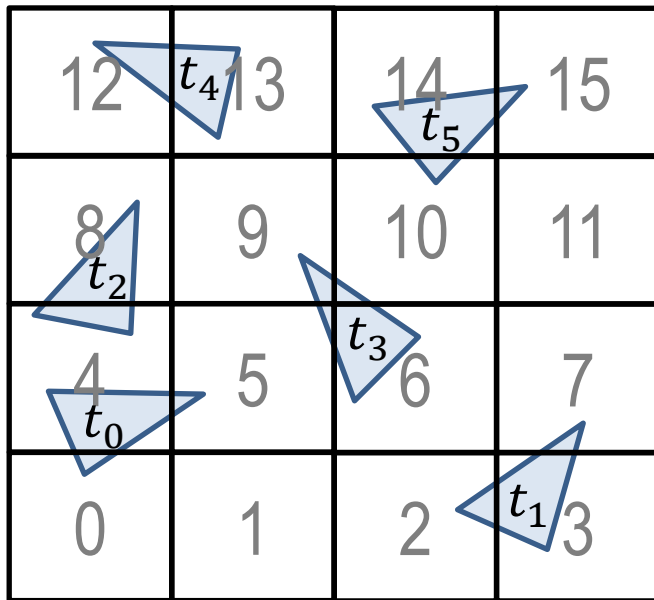


碰撞检测 – Broad phase



碰撞检测 – Broad phase

- 空间划分 – 均匀网格
- 将空间划分为均匀网格，记录每个网格中的物体
- 需要选择合适的网格大小



可能的碰撞对

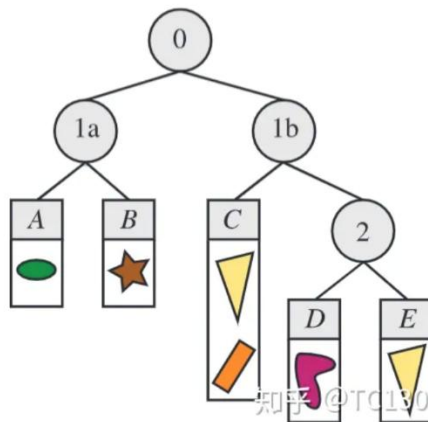
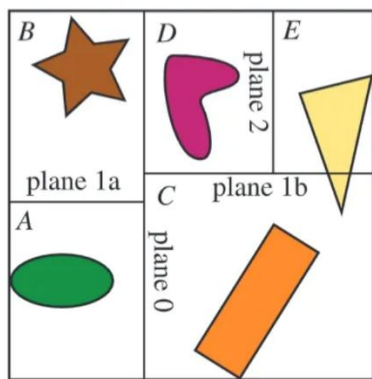
(t_0, t_3)

(t_3, t_5)

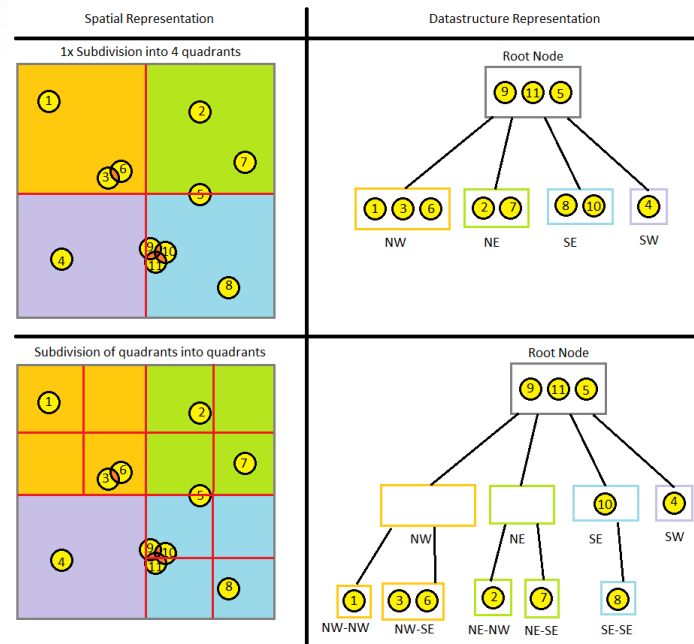
(t_0, t_2)

碰撞检测 – Broad phase

- 其他空间划分 – 空间二分树/八叉树等



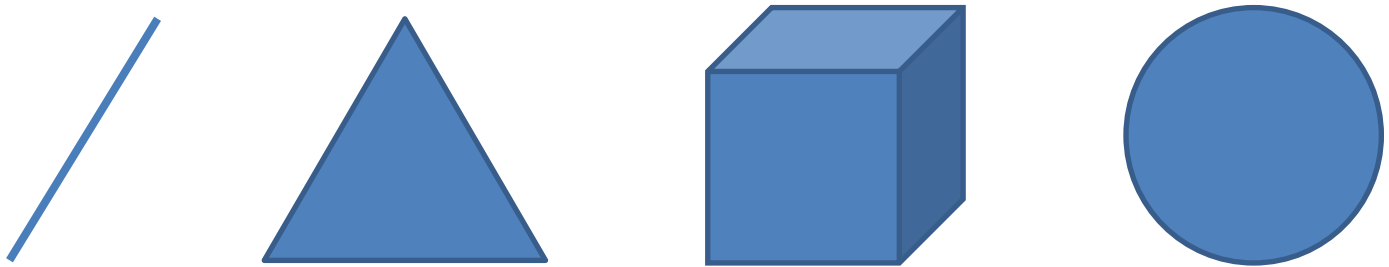
空间二分树



四叉树(2D)/八叉树(3D)

碰撞检测 – Narrow phase

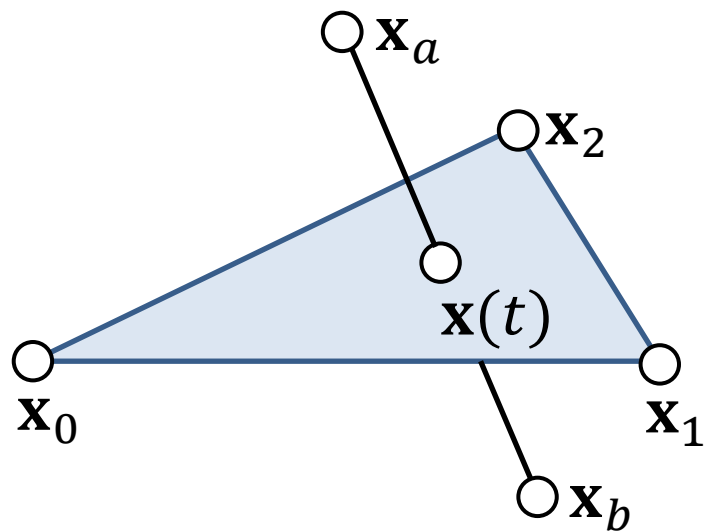
- 相交测试方法
 - 针对基本图元的求交测试精心设计算法
 - 线段、射线、球体、三角形、长方体等互相之间的相交测试



- 最基本：线段/三角形

碰撞检测算法

- 相交测试方法
 - 线段与三角形求交



$$((1 - t)\mathbf{x}_a + t\mathbf{x}_b - \mathbf{x}_0) \cdot \mathbf{x}_{10} \times \mathbf{x}_{20} = 0$$

$$t = \frac{\mathbf{x}_{0a} \cdot \mathbf{x}_{10} \times \mathbf{x}_{20}}{\mathbf{x}_{ba} \cdot \mathbf{x}_{10} \times \mathbf{x}_{20}} \quad \text{直线与平面求交}$$

如果 $t \in [0, 1]$

$$\mathbf{x}(t) = (1 - t)\mathbf{x}_a + t\mathbf{x}_b$$

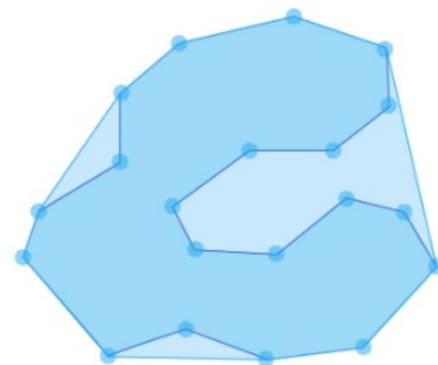
$\mathbf{x}(t)$ 在三角形内?

是

相交

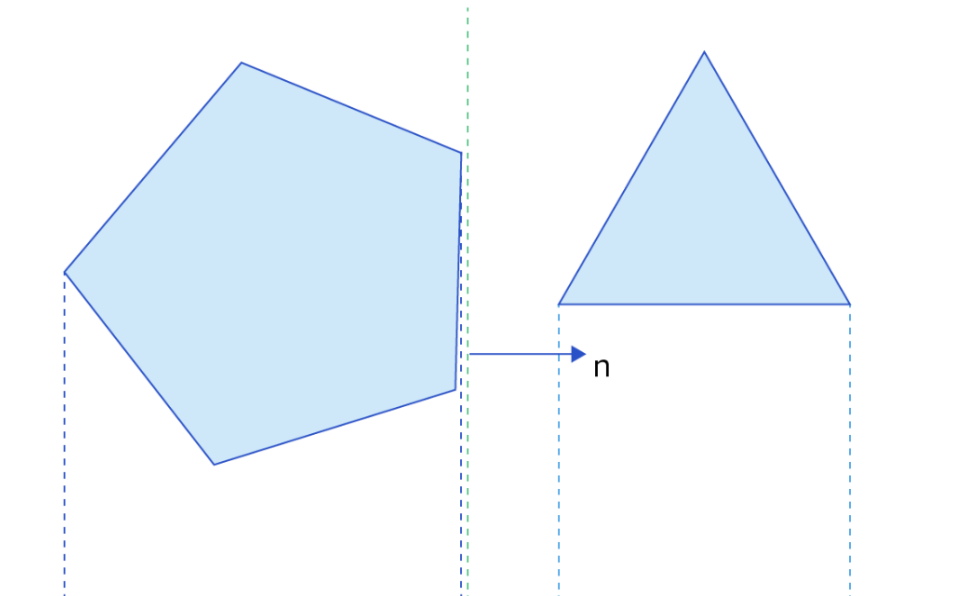
碰撞检测算法

- 一些直观的相交测试方法
 - 三角形与三角形：多次运用线段与三角形求交
 - 球与长方体：比较圆心到各平面的距离，与半径对比
 - 球与三角形：先判断和平面是否有交，再转化为2D上三角形和圆求交
- 一般几何体
 - 凹几何体：分解为凸几何体或者基本面片
 - 凸几何体：SAT算法



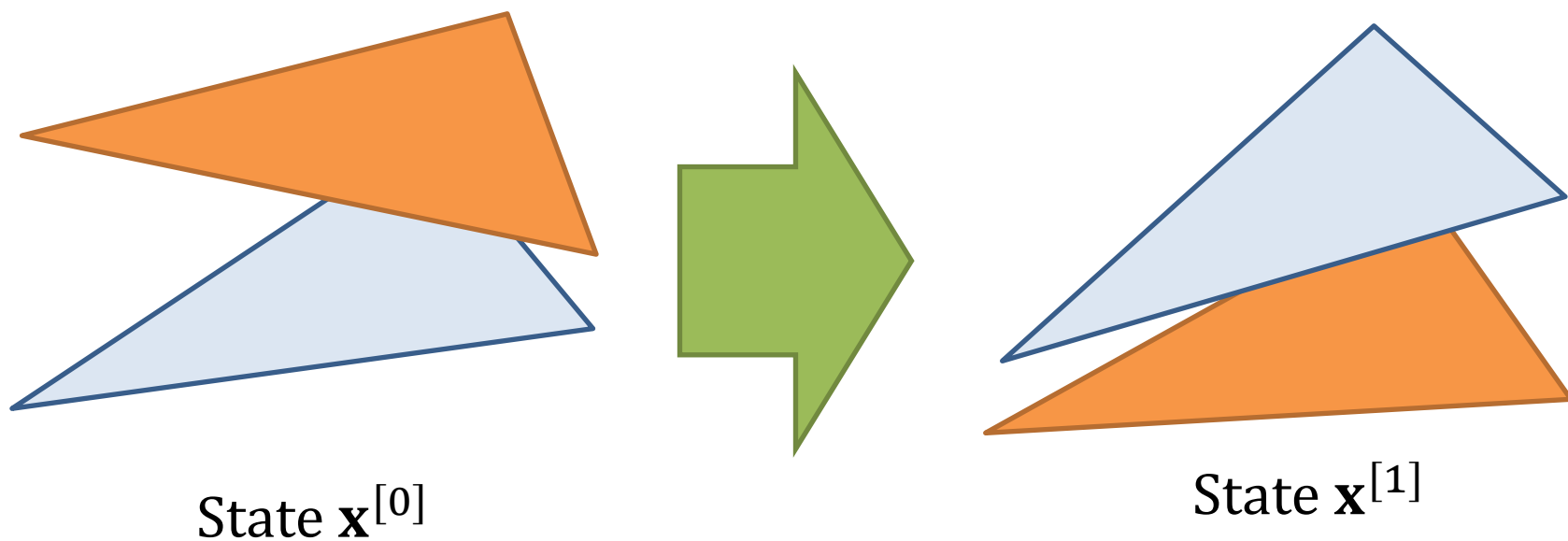
碰撞检测算法-SAT

- 两个凸几何体：
 - 对其中一个几何体遍历所有的面，检测另一个几何体的所有顶点是否都在同一侧



离散碰撞检测算法

- 在每一时间的离散点上进行静态碰撞检测
- 实现简单：应用的主流



离散碰撞检测算法

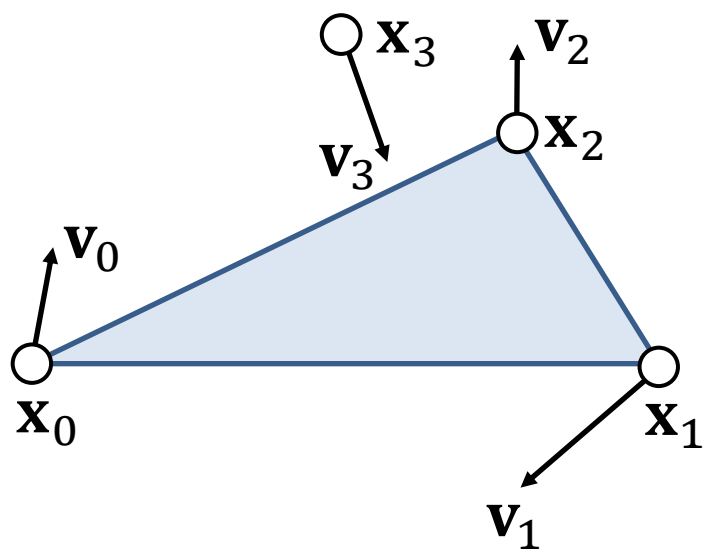
- 存在问题
 - 存在刺穿现象：通过回滚的方法才能获得第一时间的碰撞信息
 - 遗漏应该发生的碰撞
 - 缩短时间步长
 - 上述问题减弱
 - 计算量迅速上升

连续碰撞检测算法

- 计算时空扫略体进行求交
- 在连续的时间间隔内进行碰撞检测
- 在普通场合下，计算速度还太慢
- 在具有精确性要求的场合，同样的误差控制下比离散求交快
- 浮点数精度限制
- 实现复杂

连续碰撞检测算法

- 基本测试：动点 $\mathbf{x}_3$ 与动三角形 $\mathbf{x}_0\mathbf{x}_1\mathbf{x}_2$ 求交



$$\mathbf{x}_3(t) \cdot \mathbf{x}_1(t) \times \mathbf{x}_2(t) = 0 \quad \text{共平面约束}$$
$$at^3 + bt^2 + ct + d = 0 \quad \text{三次方程求解}$$

$$t \in [0, 1]$$

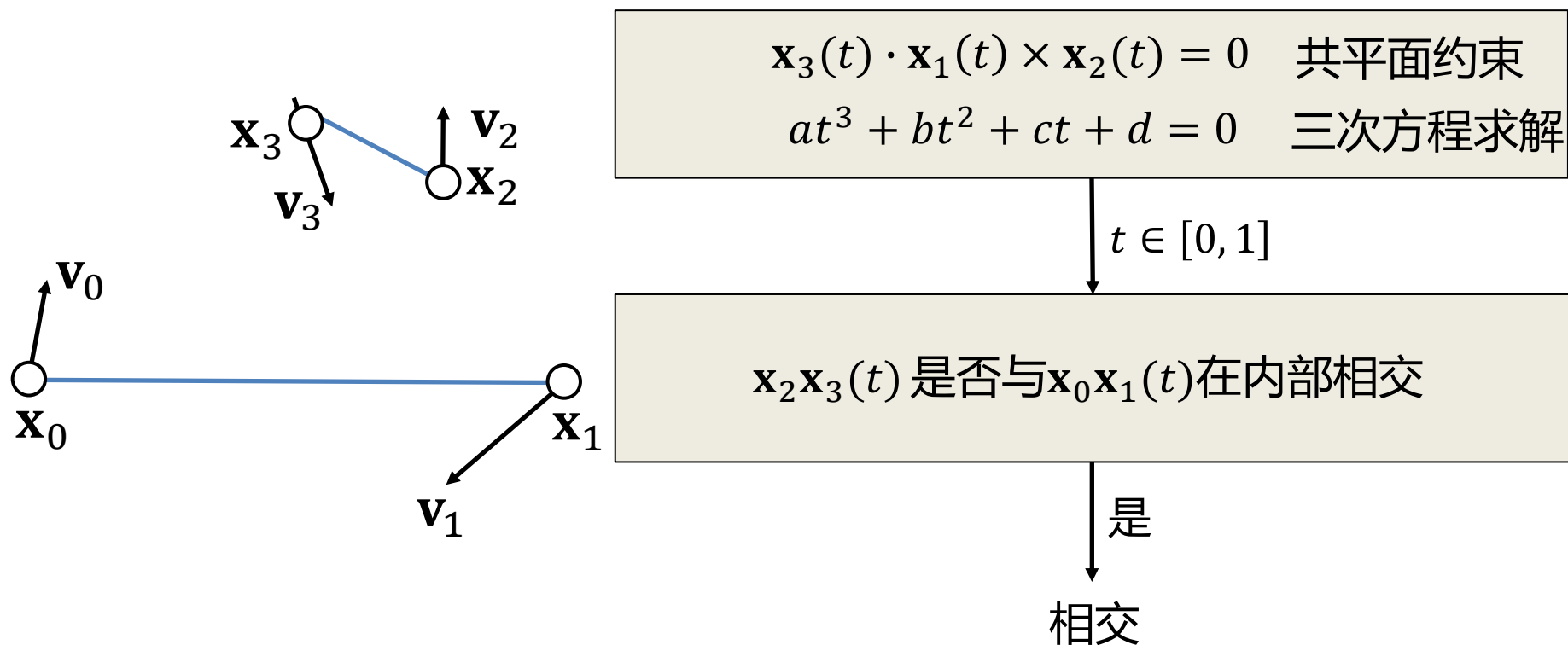
$\mathbf{x}_3(t)$ 是否在三角形 $\mathbf{x}_0\mathbf{x}_1\mathbf{x}_2(t)$ 内

是

相交

连续碰撞检测算法

- 基本测试：动线段 $\mathbf{x}_2\mathbf{x}_3$ 与动线段 $\mathbf{x}_0\mathbf{x}_1$ 求交



以下关于碰撞检测的描述中，正确的是：

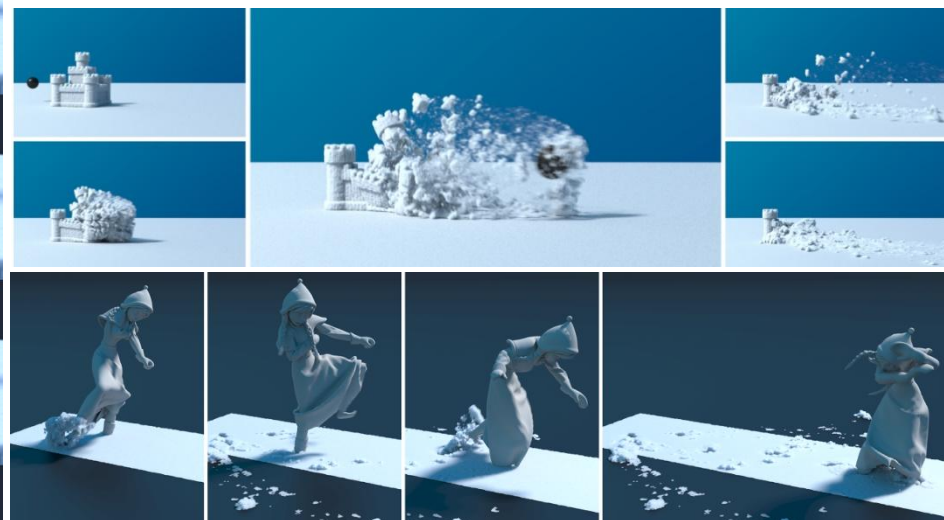
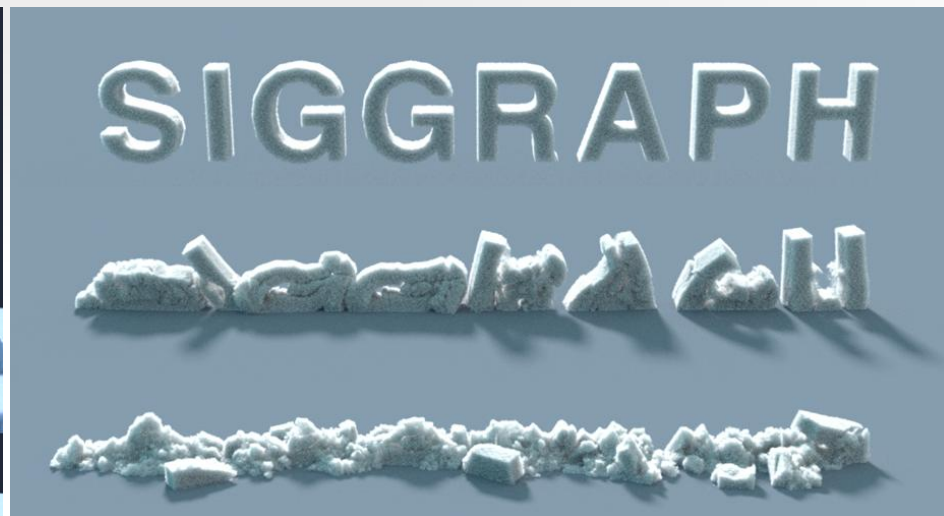
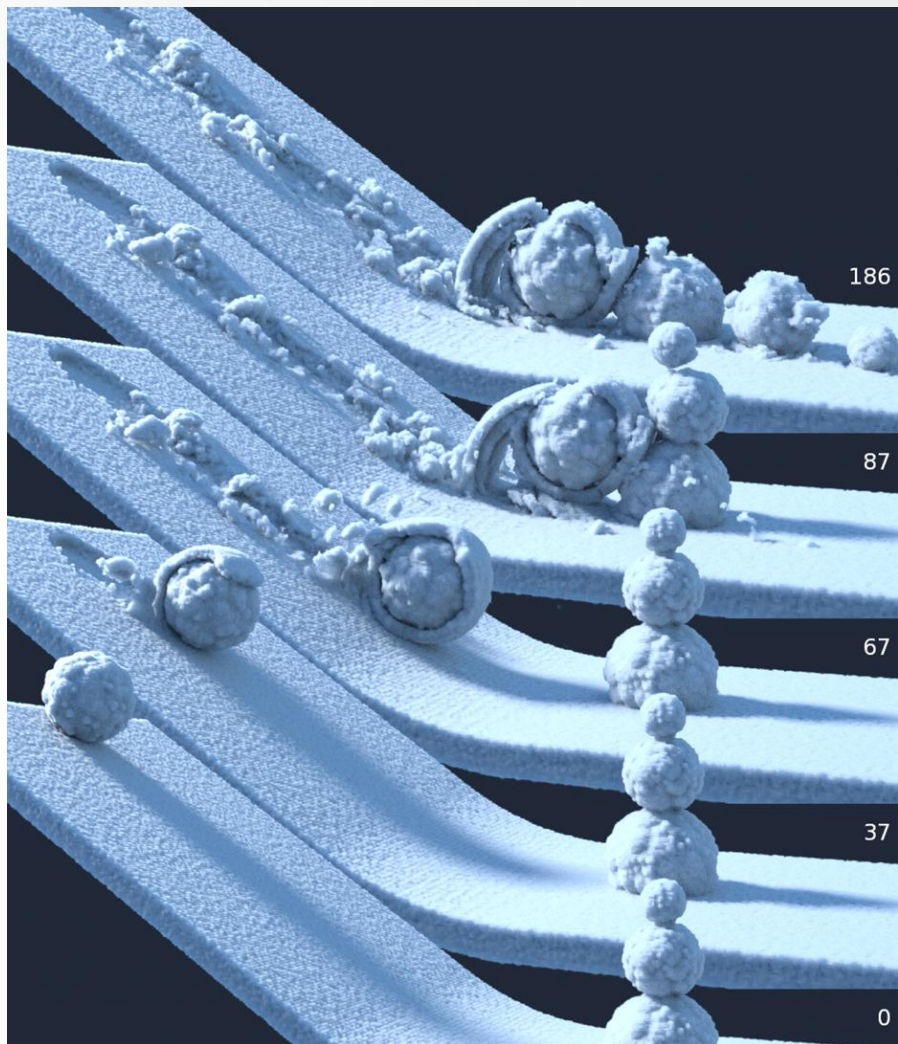
- ☒ A 若不使用任何加速方法，则随着物体数量增长，碰撞检测的计算复杂度会平方增长
- ☒ B 层次包围体和空间八叉树的实现都采用了树形数据结构
- ☒ C 离散碰撞检测算法在物体运动速度较快时更容易遗漏实际发生的碰撞

提交

Divergence-Free Smoothed Particle Hydrodynamics

Jan Bender and Dan Koschier

流体模拟效果 - 雪的模拟



流体模拟效果 - 粘稠SPH

A Unified Second-Order Accurate in Time MPM Formulation for Simulating Viscoelastic Liquids with Phase Change

Haozhe Su*[#]

Tao Xue*[#]

Chengguizi Han*

Chenfanfu Jiang[†]

Mridul Aanjaneya*

[#]Joint first authors



(no audio)

统一计算框架

- 以统一模型实时模拟更多的物理现象

Unified Particle Physics for Real-Time Applications

Miles Macklin, Matthias Müller, Nuttapong Chentanez, Tae-Yong Kim



高性能编程语言-taichi

- 嵌入Python中的高性能特定领域语言
 - 专为计算机图形应用设计，兼顾生产力、便捷性、可移植性
 - 面向数据，并行计算
 - 数据结构与计算解耦
 - 支持系数张量
 - 支持微分编程



高性能编程语言-taichi

- Taichi的安装十分简单，和安装普通Python模块一样：python3 -m pip install taichi
- 支持所有主流操作系统：Linux, Win, Mac
- 支持CPU和GPU：后端包括CPU、CUDA、OpenGL、Metal等
- 相同的一份代码可以在不同系统和后端运行

```
import taichi as ti
ti.init(arch=ti.cpu) # default backend, x64 or arm
ti.init(arch=ti.gpu) # try [cuda, metal, opengl] in order
ti.init(arch=ti.x64/arm/cuda/opengl/metal) # specific backend
```

高性能编程语言-taichi

- Taichi使用Python语法，但是有自己的数据类型和函数，以及编译器
- Taichi所有的计算通过kernel和function完成，分别用Python修饰符@ti.kernel和@ti.func修饰
- @ti.kernel中使用struct-for能够并行运行

```
@ti.kernel
def hello(i: ti.i32):
    a = 40
    print('Hello world!', a + i)

hello(2) # "Hello world! 42"
```

```
@ti.kernel
def calc() -> ti.i32:
    s = 0
    for i in range(10):
        s += i
    return s # 45
```

高性能编程语言-taichi demo

```
import taichi as ti

ti.init(arch=ti.gpu) # Run on GPU by default

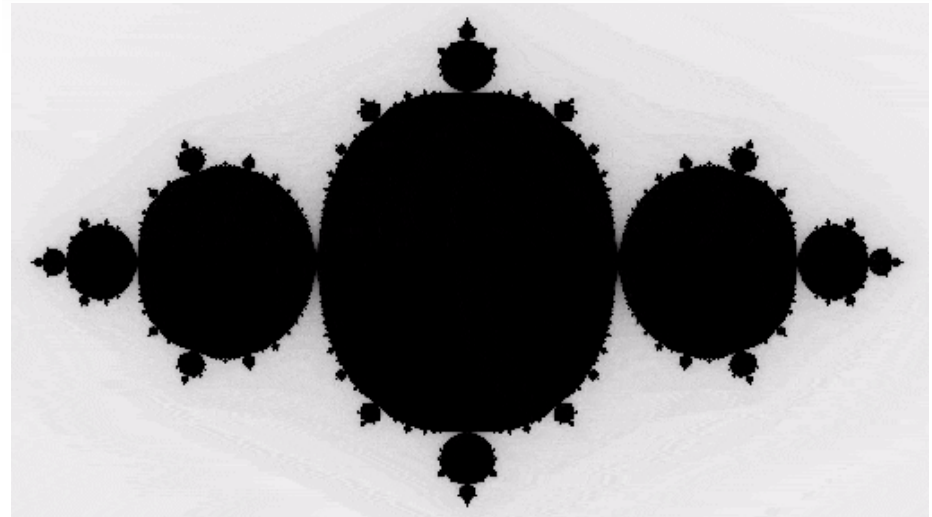
n = 320
pixels = ti.field(dtype=float, shape=(n * 2, n))

@ti.func
def complex_sqr(z):
    return ti.Vector([z[0]**2 - z[1]**2, z[1] * z[0] * 2])

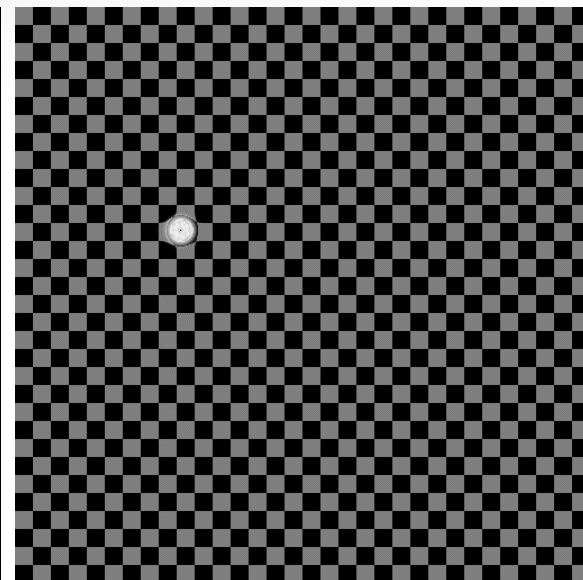
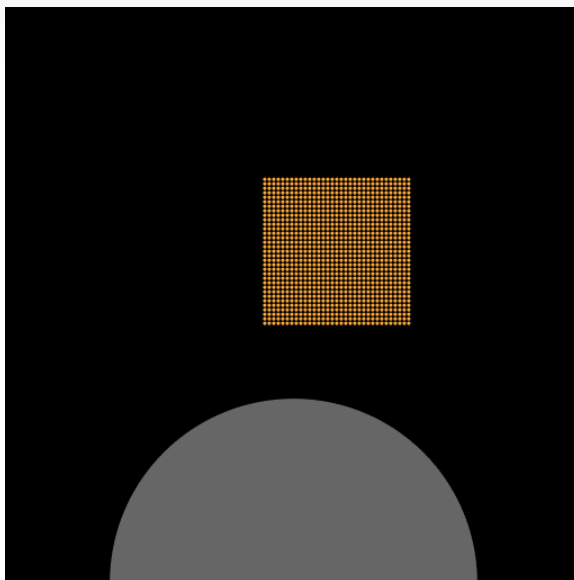
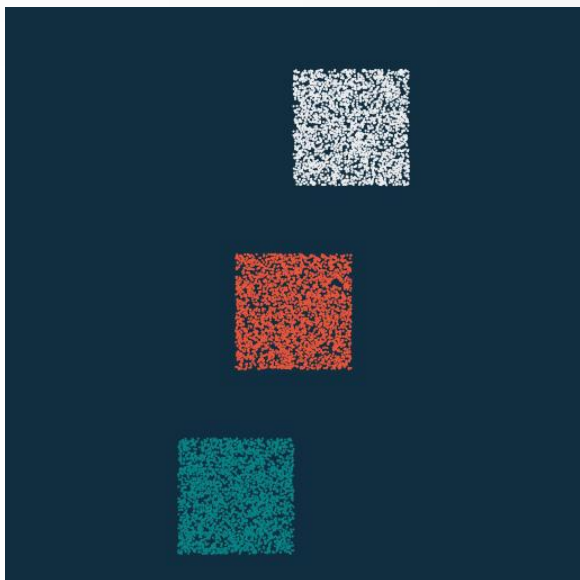
@ti.kernel
def paint(t: float):
    for i, j in pixels: # Parallized over all pixels
        c = ti.Vector([-0.8, ti.cos(t) * 0.2])
        z = ti.Vector([i / n - 1, j / n - 0.5]) * 2
        iterations = 0
        while z.norm() < 20 and iterations < 50:
            z = complex_sqr(z) + c
            iterations += 1
        pixels[i, j] = 1 - iterations * 0.02

gui = ti.GUI("Julia Set", res=(n * 2, n))

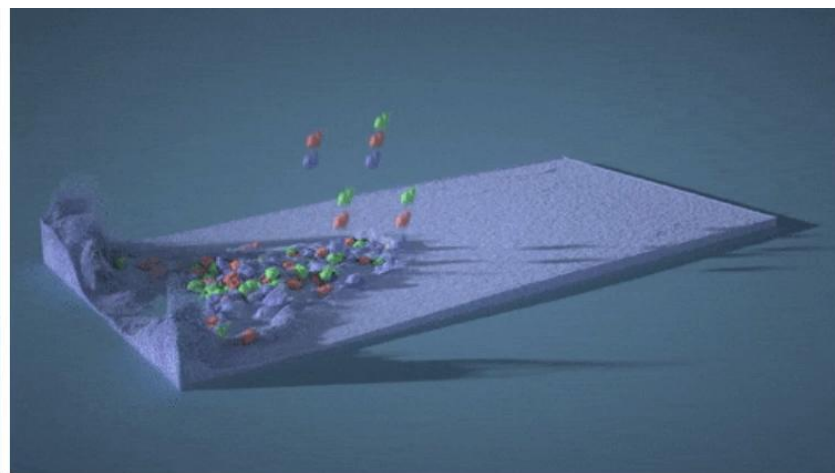
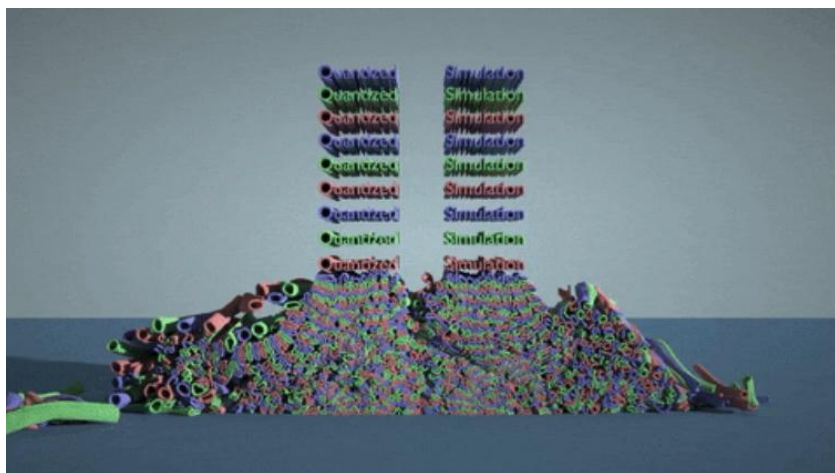
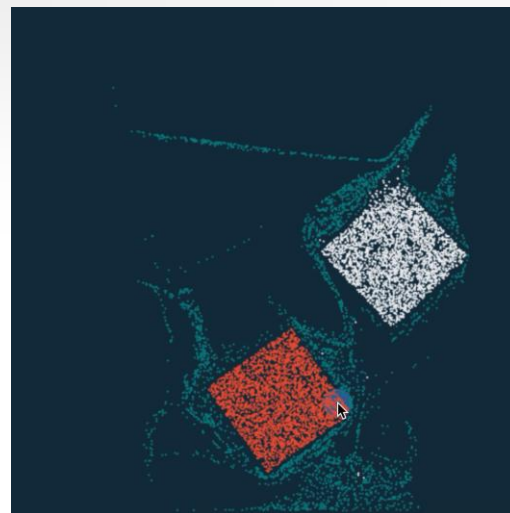
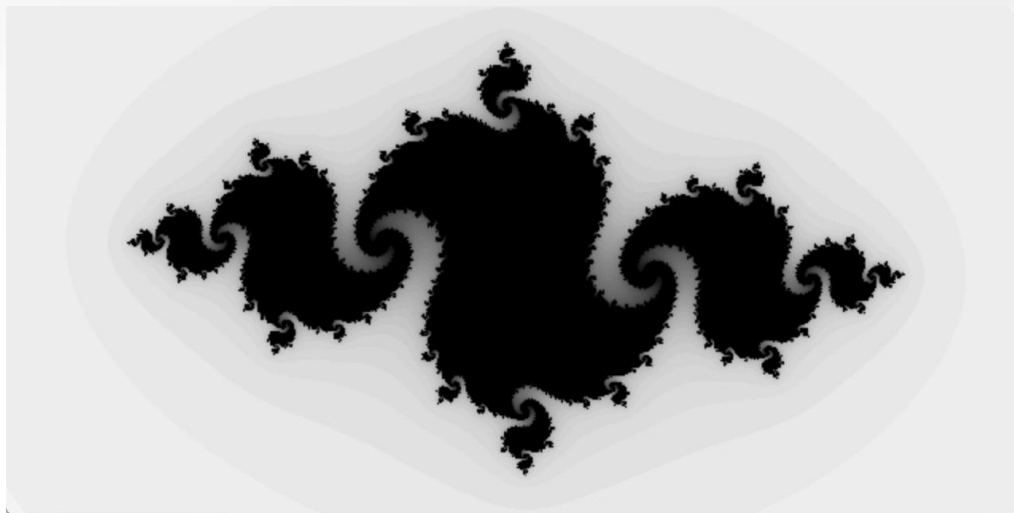
for i in range(1000000):
    paint(i * 0.03)
    gui.set_image(pixels)
    gui.show()
```



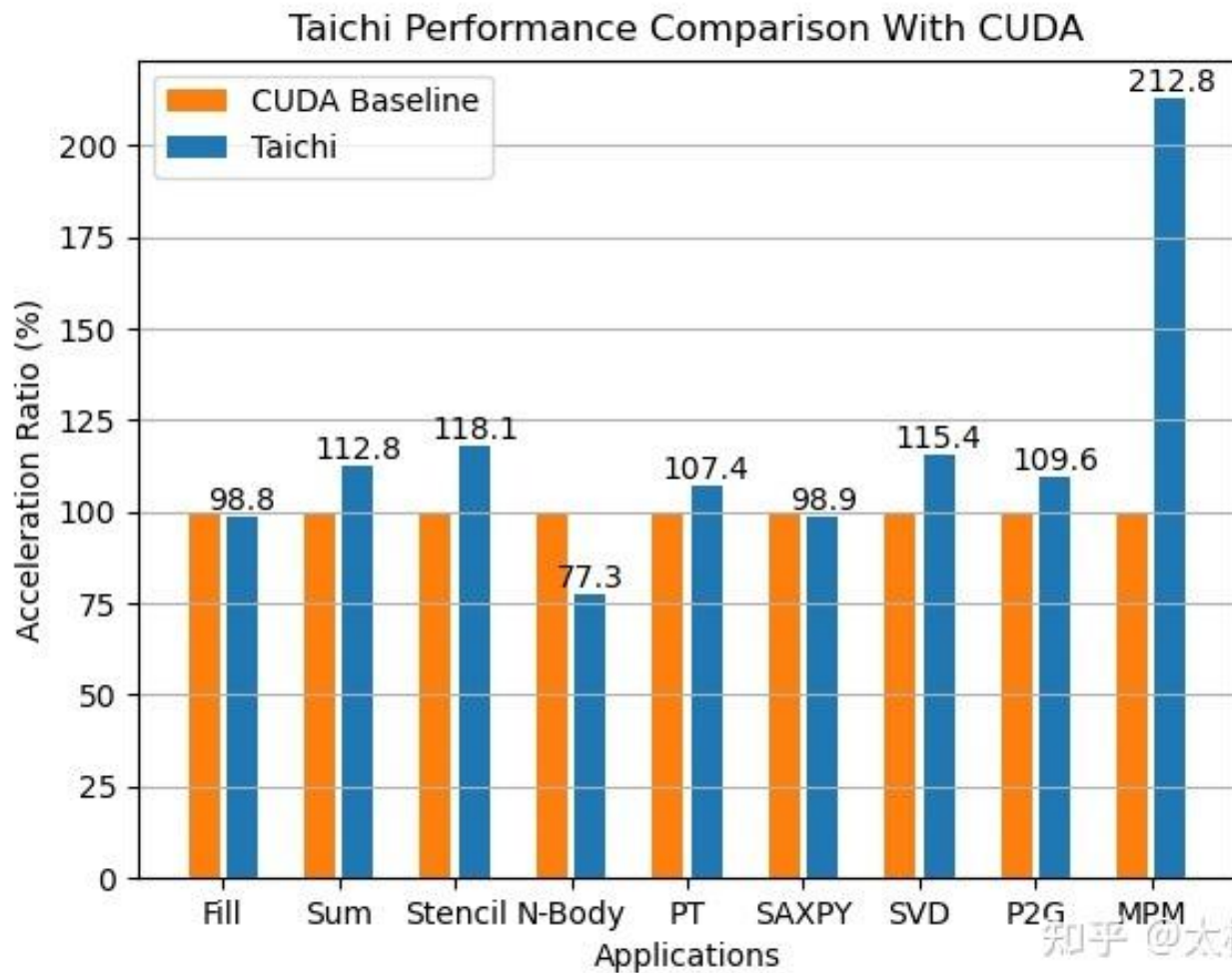
高性能编程语言-taichi demo



高性能编程语言-taichi demo



高性能编程语言-taichi



高性能编程语言-taichi应用

- 实时物理模拟
- 数值计算
- 增强现实
- 人工智能、视觉和机器人
- 电影和游戏的视觉特效
- 通用计算
- ...

高性能编程语言-taichi 学习资源

- 感兴趣的同学可以关注Taichi相关资料
- 欢迎同学们大作业选取物理模拟相关题目
- 主页: <https://github.com/taichi-dev/taichi>
- 文档: <https://taichi.readthedocs.io/en/stable/>
- 太极图形课: <https://github.com/taichiCourse01>
- Games 201课程
 - 论坛: <https://forum.taichi.graphics/>
 - 主要讨论流体模拟

物理引擎

- 什么是物理引擎
- 什么时候使用物理引擎
- 物理引擎的内容
- 物理引擎的主要技术简介
- 物理引擎的工作流程
- 物理引擎的主要产品
- 正在研究的主要问题

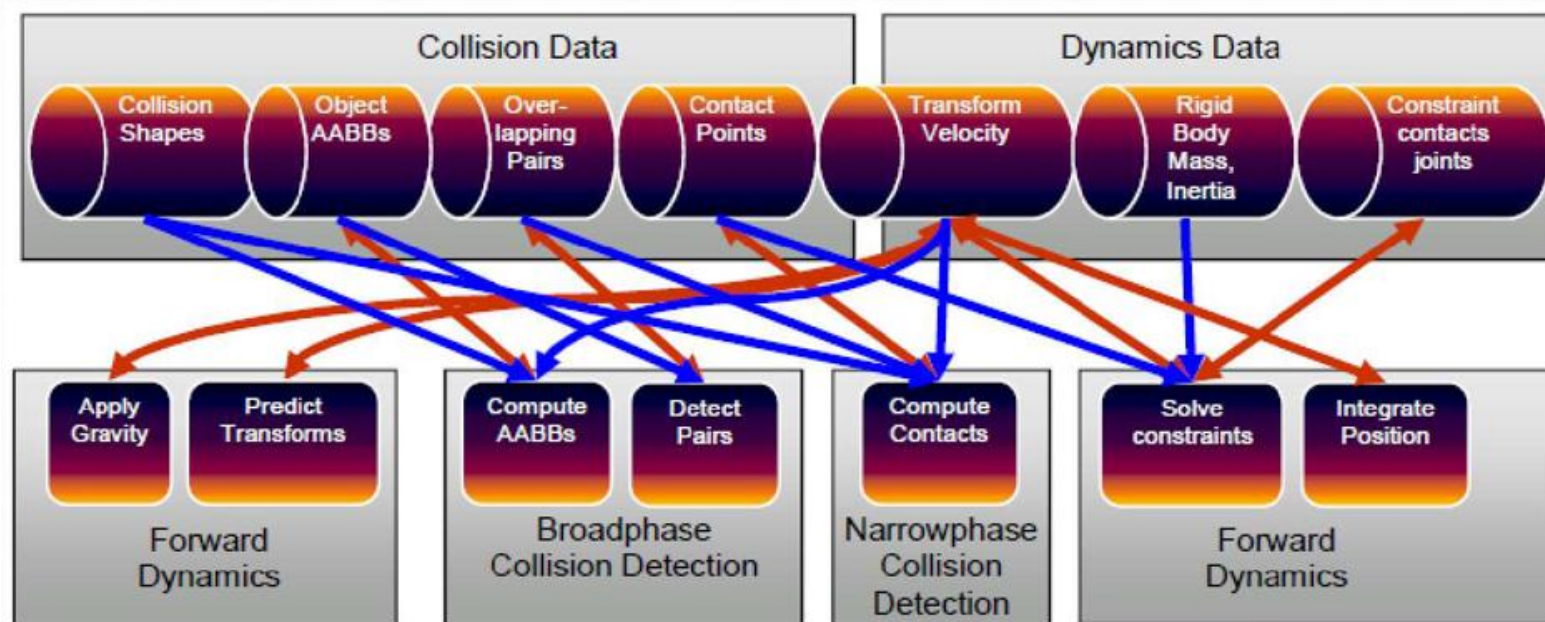
物理引擎的工作流程

- 与虚拟场景的渲染相配合，在绘制的每一帧，对具有物理属性的模拟实体进行状态与受力分析。
- 进行碰撞检测，找出运动实体间的相互约束信息，通过简化的数字物理方法计算出每个实体新的位置和状态；
- 更新实体的状态，获得新的绘制场景。

物理引擎的基本处理步骤

- 物理模拟：以**牛顿力学**或其他数学物理方法，根据实体对象的基本属性，计算出各种力对实体的作用，确定其将要发生的运动状态。
- 对每个实体对象运用向量叠加法计算合力，运用**平行轴定理**计算合力矩。
- 进行碰撞检测，根据实体的运动状态检测他们之间的交互情况。
- 进行碰撞分析，响应碰撞并进行碰撞处理。更新实体的运动状态。

Bullet引擎中的处理流程



Rigid Body Physics Pipeline

物理引擎的主要产品

- 商业物理引擎：Havok公司的Havok-Physics引擎（已被Intel收购）、NVIDIA公司的PhysX（前身为AgeiaTM公司的NovodexTM）。
- 学术研究性质的产品：Newton Game Dynamics、Copenhagen大学的OpenTissue。
- 开源形式的物理引擎产品，如Bullet、ODE、TOKAMAK。

PhysX

- NVIDIA PhysX基于NVIDIA CUDA，是实现实时物理学特效的最佳途径，这些特效包括尘土飞扬、令物体碎片四射的爆炸、生动逼真的人物动作以及衣服布料的自然下垂与撕裂等。
- PhysX技术被广泛应用于150多个游戏中，其注册用户数量已超过10,000名。这项技术在索尼的Playstation3、微软的Xbox360、任天堂的Wii以及个人计算机上均得到了良好的支持，把游戏推向全新的境界。

PhysX的主要特色

- 爆炸引起的烟尘和随之产生的碎片
- 复杂、连贯的几何学计算使人物的动作和互动更加逼真
- 具有可缩放、复杂、逼真、高度互动的特性
- 布纹的编织和撕裂效果非常自然
- 运动物体周围烟雾翻腾
- https://v.youku.com/v_show/id_XMTQ3MjE0MjQyOA==

PhysX的运用效果

- 布料

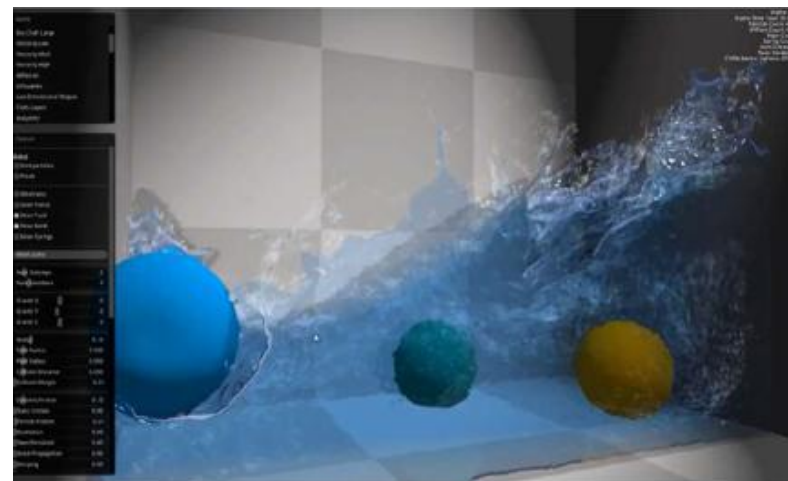


- 游戏



PhysX的运用效果

- 流体



Havok

- 全称为HavokGameDynamicsSDK，译作Havok游戏动力开发包，与PhysX相提并论的Havok物理模拟引擎，只是Havok引擎中的一部分而已。
- HavokPhysics基于CPU计算，针对多核多线程CPU进行了特别优化，可以提供快速高效率的物理模拟计算，是业界功能最全面的物理仿真解决方案，已应用在诸如实时碰撞计算、动力学约束求解、车辆综合解决方案等多个领域。
- Havok对PS2、XBOX、GameCube、PC多种游戏平台都有支持，成功的案例有：：《星际争霸2》、《暗黑破坏神3》、《战地：叛逆连队2》、《上古卷轴4》、《马克思佩恩2》、《光晕3》、《半条命2》、《细胞分裂》、《生化危机5》等等。

Havok的组成部件

- HavokFX, 针对爆炸效果等大计算量的物理模拟。
- HavokAnimation, 动作引擎。是一种高效灵活的动作开发工具。
- HavokBehavior, 行为引擎。它可以让游戏中的虚拟人物学会新的行为、动作、战术。
- HavokCloth, 布料模拟引擎。能模拟一切柔性物体。如衣服, 裙子, 斗篷, 外套、头发、尾巴、旗帜、横幅、窗帘、植物等等。
- HavokDestruction, 刚体破坏引擎。带来前所未有的高范围破坏, 可完全毁坏游戏内的所有物体(载具、建筑、桥梁、树木.....)和部分地形。
- HavokAI, 人工智能引擎。可以更容易的设计出更出色、聪明的游戏角色。

Havok效果演示

- 一个怪物及其身上的衣服，需要六个线程在Core i7处理器上同时运行，同时渲染三四十个怪物之后则能用光几乎全部处理器核心资源。显然，不同于NVIDIA PhysX依赖显卡运算，Intel Havok主要是利用多核心处理器来执行。
- 布料模拟演示中，模特身着的红裙在Havok物理引擎关闭的时候异常生硬，毫无质感，打开之后则是摇曳飘逸，甚至可以有16位模特同时翩翩起舞，每个人的裙子都是即时动态生成的。
- 最后一段颇为类似NVIDIA Fermi的物理演示，也是一列火车运行在高架桥上，可以对其进行轰击，火车和铁桥都会随之灰飞烟灭。



Havok效果演示

- 刚体破坏



- 汽车



游戏



Havok效果演示

- <https://tv.sohu.com/v/dXMvMjA0NzE1NTYvNTI2OTM3OTEuc2h0bWw=.html>
- https://v.youku.com/v_show/id_XNzk1MDY4MzY4.html?from=s1.8-1-2.999&f=19355091
- https://v.youku.com/v_show/id_XMjA4NTQwODAw.html

Bullet

- Bullet据称是游戏世界占有率排名第三的物理引擎，也是前几大引擎目前唯一能够找到的支持iPhone，开源，免费的物理引擎。
- Bullet是AMD开放物理计划成员之一。在与AMD合作后，此引擎可以透过OpenCL或者DirectCompute使用GPU完成物理模拟计算。

Bullet

- Bullet是一个跨平台的物理模拟计算引擎。支持Windows、Linux、MAC、Playstation3、XBOX360、NintendoWii。Bullet也整合到了Maya和Blender3D中。
- Bullet广泛应用于游戏开发和电影制作中。其中游戏包括汽车总动员2、侠盗猎车4、特技摩托赛HD等；电影有全民超人、大侦探福尔摩斯、闪电狗等等。

Bullet引擎演示



Bullet物理引擎的刚体破坏演示

<https://tv.sohu.com/v/dXMvNjMyNjUwODkvMjQxMjI4ODYuc2h0bWw=.html>

Unigine

- 该引擎包含了逼真的三维渲染，强大的物理模块，具有非常丰富的图书馆导向的脚本系统，全功能的GUI模块，声音子系统，以及灵活的工具，高效率 and 良好架构的框架，支持多核系统，是一个高度可扩展的解决方案，对其中的多平台类型游戏的影响颇多。



高度优化的渲染确保出色的图像质量和强大的性能

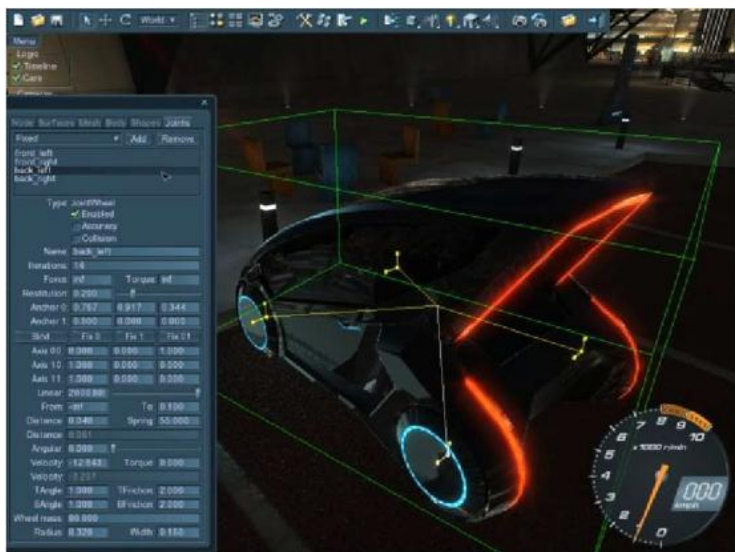


支持各种声音、音乐和3D音效的虚拟环境

Unigine

- 该引擎包含了逼真的三维渲染，强大的物理模块，具有非常丰富的图书馆导向的脚本系统，全功能的GUI模块，声音子系统，以及灵活的工具，高效率和良好架构的框架，支持多核系统，是一个高度可扩展的解决方案，对其中的多平台类型游戏的影响颇多。

支持脚本语言，具有高效的字节
码编译器和优化恢复



支持全场景序列化，让你在
倍感真实的物理世界中互动



Unigine引擎物理破坏效果演示



Genesis – 生成式物理引擎

- Genesis 是专为 *机器人/嵌入式 AI/物理 AI* 应用设计的通用物理平台，集成了以下核心功能：
- **通用物理引擎**: 从底层重建, 支持多种材料和物理现象模拟
- **机器人模拟平台**: 轻量、高速、Python友好的开发环境
- **真实感渲染**: 内置光线追踪渲染系统
- **生成数据引擎**: 自然语言驱动的多模态数据生成

```
In [1]: import genesis as gs  
  
In [2]: gs.generate("A water droplet drops onto a beer bottle and then  
         slowly slides down along the bottle surface")
```

Camera View A

```
In [2]: gs.generate("...; camera  
         revolves around the bottle  
         while gradually moving  
         downward.")
```

Camera View B

```
In [2]: gs.generate("...; a close-up  
         shot from above, tracking the  
         motion of the droplet.")
```

Camera View C

```
In [2]: gs.generate("...; camera is  
         positioned on the right side  
         of the scene and tracks the  
         droplet motion.")
```


正在研究的主要问题

- 首先是碰撞检测和碰撞分析中的稳定性问题。目前商业引擎和非商业引擎在效率上区别不是很大，但是商业引擎的稳定性要好很多。
- 其次是动力学模型的求解问题。目前最常用的、精度最高的是基于约束的方法，它需要求解由约束方程构成的线性完备性问题(LCP)。例如在HavokTM,PhysXTM/NovodexTM,ODE,Bullet等物理引擎中都采用这样的思路。如何使求解速度更快代价更小是研究的方向。
- 再有是物理模拟的并行计算问题。现有引擎因为针对的应用领域大多是游戏动画方面，场景相对来说比较小，对并行计算的需求并不大。基于多核并行或是分布式网络的较为通用的并行模拟仿真应该是大规模应用研究和开发的方向。

存在的问题

- 处理大规模复杂场景的能力不足
- 基于图象的碰撞检测算法的精度不高
- 离散碰撞检测算法的刺穿和遗漏问题
- 面向特定领域需针对其研究高效碰撞检测算法

参考资料

- Gatech 刚体模拟课程网站:
 - https://www.cc.gatech.edu/classes/AY2012/cs4496_spring/Calendar.html
- 美国UNC大学Ming Lin教授课程：
 - <https://www.cs.unc.edu/~lin/COMP259-S06/>
- Markus Ihmsen, Jens Orthmann, Barbara Solenthaler, Andreas Kolb, Matthias Teschner, SPH Fluids in Computer Graphics, Eurographics 2014
- Markus Becker, Matthias Teschner, Weakly compressible SPH for free surface flows, ACM Siggraph Symposium on Computer Animation 2007
- Yongning Zhu, Robert Bridson, Animating Sand as a Fluid, ACM Siggraph 2005
- Su, Haozhe and Xue, Tao and Han, Chengguizi and Jiang, Chenfanfu and Aanjaneya, Mridul. A unified second-order accurate in time MPM formulation for simulating viscoelastic liquids with phase change, ACM Siggraph 2021
- Robert Bridson, Fluid Simulation for Computer Graphics
- Xiao Yan, Yun-Tao Jiang, Chen-Feng Li, Ralph R. Martin, and Shi-Min Hu. 2016. Multiphase SPH simulation for interactive fluids and solids. ACM Trans. Graph. 35, 4, Article 79 (July 2016).



THANKS!

清华大学计算机系

张松海

shz@Tsinghua.edu.cn